

Aprendizaje y Clasificación Automática con R

Un enfoque práctico con datos abiertos reales

MI. Gilberto Rodríguez

2026-07-09

Índice general

Presentación	1
Objetivo general	1
Público al que va dirigido	1
Software utilizado	1
Estructura inicial del libro	1
Nota para el lector	2
1. ¿Qué es el aprendizaje automático?	3
1.1. Objetivos del capítulo	3
1.2. Introducción	3
1.3. Programación tradicional contra aprendizaje automático	3
1.4. Variables predictoras y variable respuesta	4
1.5. Tipos principales de aprendizaje automático	5
1.6. Notación básica para aprendizaje automático	5
1.7. Algoritmo general de aprendizaje supervisado	5
1.8. Aprendizaje supervisado	6
1.9. Clasificación	6
1.10. Regresión	6
1.11. Aprendizaje no supervisado	6
1.12. Flujo general de un proyecto de aprendizaje automático	6
1.13. Primer ejemplo visual en R	7
1.14. División entre entrenamiento y prueba	7
1.15. Primer clasificador sencillo basado en una regla	8
1.16. Conclusión	9
2. Preparación de datos reales	11
2.1. Objetivos del capítulo	11
2.2. Introducción	11
2.3. Datos crudos y datos preparados	11
2.4. Representación matemática de una base preparada	11
2.5. Algoritmo general de preparación de datos	12
2.6. Conjunto de datos de trabajo: ATUS	12
2.7. Crear la carpeta de datos	12
2.8. Cargar paquetes	12
2.9. Leer el archivo CSV	13
2.10. Primer vistazo a los datos	13

2.11. Estandarizar nombres de variables	14
2.12. Variables de interés	15
2.13. Diccionario inicial de variables	15
2.14. Revisión de la variable CLASACC	16
2.15. Crear una variable respuesta	16
2.16. Eliminar registros sin variable respuesta	17
2.17. Convertir variables categóricas	17
2.18. Revisión de datos faltantes	18
2.19. Selección final de variables para el primer modelo	18
2.20. Guardar la base preparada	19
2.21. Resumen del capítulo	19
3. Análisis exploratorio de datos	21
3.1. Objetivos del capítulo	21
3.2. Introducción	21
3.3. Fundamento matemático del análisis exploratorio	21
3.4. Algoritmo general	21
3.5. Cargar paquetes y base	22
3.6. Preparar variables para el análisis exploratorio	22
3.7. Primer vistazo a la base	23
3.8. Distribución de la variable respuesta	24
3.9. Accidentes por mes	25
3.10. Accidentes por hora del día	26
3.11. Accidentes por día de la semana	27
3.12. Accidentes por tipo de accidente	29
3.13. Accidentes por causa presunta	30
3.14. Relación entre tipo de accidente y presencia de víctimas	31
3.15. Relación entre hora y presencia de víctimas	32
3.16. Relación entre día de la semana y presencia de víctimas	33
3.17. Tabla resumen por clase	34
3.18. Conclusión	34
4. Regresión logística para clasificación	35
4.1. Objetivos del capítulo	35
4.2. Introducción	35
4.3. La función logística	35
4.4. Fundamento matemático	35
4.5. Algoritmo de regresión logística	36
4.6. Visualización de la función logística	36
4.7. Cargar paquetes y datos	37
4.8. Preparar variables para el modelo	38
4.9. División en entrenamiento y prueba	38
4.10. Ajustar el modelo	39
4.11. Predicción de probabilidades	39
4.12. De probabilidades a clases	40
4.13. Matriz de confusión	40
4.14. Métricas de evaluación	40
4.15. Cambiar el punto de corte	41

4.16. Distribución de probabilidades predichas	42
4.17. Odds ratios	43
4.18. Conclusión	43
5. k vecinos más cercanos, k-NN	45
5.1. Objetivos del capítulo	45
5.2. Introducción	45
5.3. Idea intuitiva del método	45
5.4. Ejemplo sencillo con puntos	45
5.5. Distancia euclidiana	46
5.6. Ejemplo de cálculo de distancia	47
5.7. Estandarización de variables	47
5.8. Algoritmo k-NN	47
5.9. Cargar paquetes y datos	48
5.10. Selección de una muestra	48
5.11. Preparar variables	49
5.12. Crear variables dummy	49
5.13. División en entrenamiento y prueba	50
5.14. Estandarización	50
5.15. Modelo k-NN con $k = 5$	50
5.16. Matriz de confusión	51
5.17. Métricas de evaluación	51
5.18. Comparación con distintos valores de k	52
5.19. Comparación conceptual con regresión logística	53
5.20. Conclusión	53
6. Árboles de decisión para clasificación	55
6.1. Objetivos del capítulo	55
6.2. Introducción	55
6.3. Idea intuitiva	55
6.4. Reglas tipo si-entonces	56
6.5. Fundamento matemático	56
6.6. Ejemplo sencillo del índice de Gini	57
6.7. Algoritmo general	57
6.8. Cargar paquetes y datos	58
6.9. Usar la base completa	58
6.10. Preparar variables	58
6.11. División en entrenamiento y prueba	59
6.12. Ajustar el árbol de decisión	60
6.13. Tabla de complejidad	60
6.14. Importancia de variables	61
6.15. Gráfica opcional del árbol	61
6.16. Predicción sobre el conjunto de prueba	62
6.17. Matriz de confusión	62
6.18. Métricas de evaluación	62
6.19. Probabilidades predichas	63
6.20. Resumen de nodos del árbol	64
6.21. Comparación conceptual con otros métodos	64

6.22. Ventajas y desventajas	64
6.22.1. Ventajas	64
6.22.2. Desventajas	65
6.23. Conclusión	65

Presentación

Este libro introduce el aprendizaje automático (*Machine Learning*) mediante ejemplos claros, didácticos y prácticos desarrollados principalmente en R.

La intención no es escribir un tratado matemático extenso, sino construir una guía aplicada para aprender haciendo con datos abiertos reales. En esta primera versión trabajaremos con datos de accidentes de tránsito de México y los usaremos como hilo conductor para explicar preparación de datos, análisis exploratorio y modelos de clasificación.

Objetivo general

Desarrollar competencias básicas e intermedias en aprendizaje automático mediante el análisis, modelado e interpretación de datos reales.

Público al que va dirigido

Este libro está dirigido a estudiantes de ingeniería, ciencias, ciencia de datos, estadística aplicada y áreas afines que desean iniciar en aprendizaje automático con un enfoque práctico.

Software utilizado

Los ejemplos se desarrollan principalmente en R, usando RStudio y Quarto. También se mencionan herramientas útiles para publicar y compartir resultados:

- RStudio
- Consola de R
- Google Colab
- GitHub
- GitHub Pages
- Quarto Pub

Estructura inicial del libro

1. Introducción al aprendizaje automático.
2. Preparación de datos reales.
3. Análisis exploratorio de datos.
4. Regresión logística.
5. k vecinos más cercanos.

Nota para el lector

Los capítulos incluyen tres elementos: fundamento conceptual, código reproducible en R e interpretación didáctica de los resultados.

Capítulo 1

¿Qué es el aprendizaje automático?

1.1. Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de:

- Explicar qué es el aprendizaje automático.
- Diferenciar entre programación tradicional y aprendizaje automático.
- Identificar problemas de clasificación, regresión y agrupamiento.
- Reconocer las etapas principales de un proyecto de aprendizaje automático.
- Reconocer la notación matemática básica usada en aprendizaje automático.

1.2. Introducción

El aprendizaje automático, también conocido como *Machine Learning*, es una rama de la inteligencia artificial que permite construir modelos capaces de aprender patrones a partir de datos.

En la programación tradicional, una persona define reglas. En aprendizaje automático, en cambio, proporcionamos ejemplos para que un algoritmo aprenda una relación entre variables de entrada y una salida esperada.

Aprender Machine Learning haciendo proyectos con datos reales.

1.3. Programación tradicional contra aprendizaje automático

Supongamos que queremos clasificar un día como de contaminación alta o baja usando una regla fija basada en PM10.

```
pm10 <- 85

if (pm10 > 75) {
  print("Contaminación alta")
} else {
  print("Contaminación baja")
}
```

```
[1] "Contaminación alta"
```

Explicación del código.

Se define el valor de PM10 y se aplica una regla condicional. Si `pm10` es mayor que 75, R imprime “Contaminación alta”; si no, imprime “Contaminación baja”.

Interpretación del resultado.

Este ejemplo muestra programación tradicional: la decisión depende de una regla escrita manualmente. En aprendizaje automático, la regla no se escribe directamente; se aprende a partir de datos.

Ahora construyamos una pequeña base de ejemplo.

```
datos <- data.frame(
  dia = 1:12,
  pm10 = c(42, 88, 55, 120, 63, 95, 38, 110, 72, 130, 47, 99),
  temperatura = c(25, 28, 22, 30, 24, 29, 21, 31, 27, 32, 23, 30),
  humedad = c(40, 35, 60, 30, 55, 33, 65, 29, 42, 28, 62, 34),
  clase = c("Baja", "Alta", "Baja", "Alta", "Baja", "Alta",
            "Baja", "Alta", "Baja", "Alta", "Baja", "Alta")
)

datos
```

	dia	pm10	temperatura	humedad	clase
1	1	42	25	40	Baja
2	2	88	28	35	Alta
3	3	55	22	60	Baja
4	4	120	30	30	Alta
5	5	63	24	55	Baja
6	6	95	29	33	Alta
7	7	38	21	65	Baja
8	8	110	31	29	Alta
9	9	72	27	42	Baja
10	10	130	32	28	Alta
11	11	47	23	62	Baja
12	12	99	30	34	Alta

Explicación del código.

Se crea una tabla con 12 días. Cada fila contiene variables ambientales (`pm10`, `temperatura` y `humedad`) y una variable de salida llamada `clase`.

Interpretación del resultado.

La tabla representa ejemplos históricos. En aprendizaje supervisado, estos ejemplos permiten aprender una relación entre variables predictoras y una clase conocida.

1.4. Variables predictoras y variable respuesta

La **variable respuesta** es la variable que queremos predecir. En el ejemplo anterior es `clase`.

Las **variables predictoras** son las variables que usamos como entrada para el modelo. En el ejemplo son `pm10`, `temperatura` y `humedad`.

1.5. Tipos principales de aprendizaje automático

En una primera aproximación podemos hablar de tres grandes tipos:

1. Aprendizaje supervisado.
2. Aprendizaje no supervisado.
3. Aprendizaje por refuerzo.

Este libro se concentrará principalmente en los dos primeros.

1.6. Notación básica para aprendizaje automático

Supongamos que tenemos n observaciones y p variables predictoras. Podemos representar los datos mediante una matriz:

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$

donde x_{ij} representa el valor de la variable j en la observación i .

La variable respuesta se representa como:

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

El objetivo general del aprendizaje supervisado es encontrar una función f que permita aproximar la respuesta:

$$\hat{y} = f(x_1, x_2, \dots, x_p)$$

1.7. Algoritmo general de aprendizaje supervisado

Algoritmo: Aprendizaje supervisado

Entrada:

- Matriz de datos X
- Variable respuesta Y
- Algoritmo de aprendizaje

Proceso:

1. Dividir los datos en entrenamiento y prueba.
2. Usar los datos de entrenamiento para aprender una función f .
3. Usar la función f para predecir nuevas respuestas.

4. Comparar las predicciones contra los valores reales.
5. Calcular métricas de desempeño.
6. Interpretar los resultados.

Salida:

- Modelo entrenado
- Predicciones
- Métricas de evaluación

1.8. Aprendizaje supervisado

El aprendizaje supervisado se utiliza cuando tenemos una variable respuesta conocida. Dentro de este tipo de aprendizaje hay dos tareas principales: **clasificación** y **regresión**.

1.9. Clasificación

La clasificación se utiliza cuando la variable respuesta es categórica. Algunos ejemplos son alta o baja contaminación, accidente con víctimas o solo daños, tumor benigno o maligno, y correo deseado o no deseado.

1.10. Regresión

La regresión se utiliza cuando la variable respuesta es numérica, por ejemplo: concentración de PM10, temperatura, ventas o número de accidentes.

1.11. Aprendizaje no supervisado

El aprendizaje no supervisado se utiliza cuando no tenemos una variable respuesta conocida. En lugar de predecir una etiqueta, buscamos patrones ocultos en los datos.

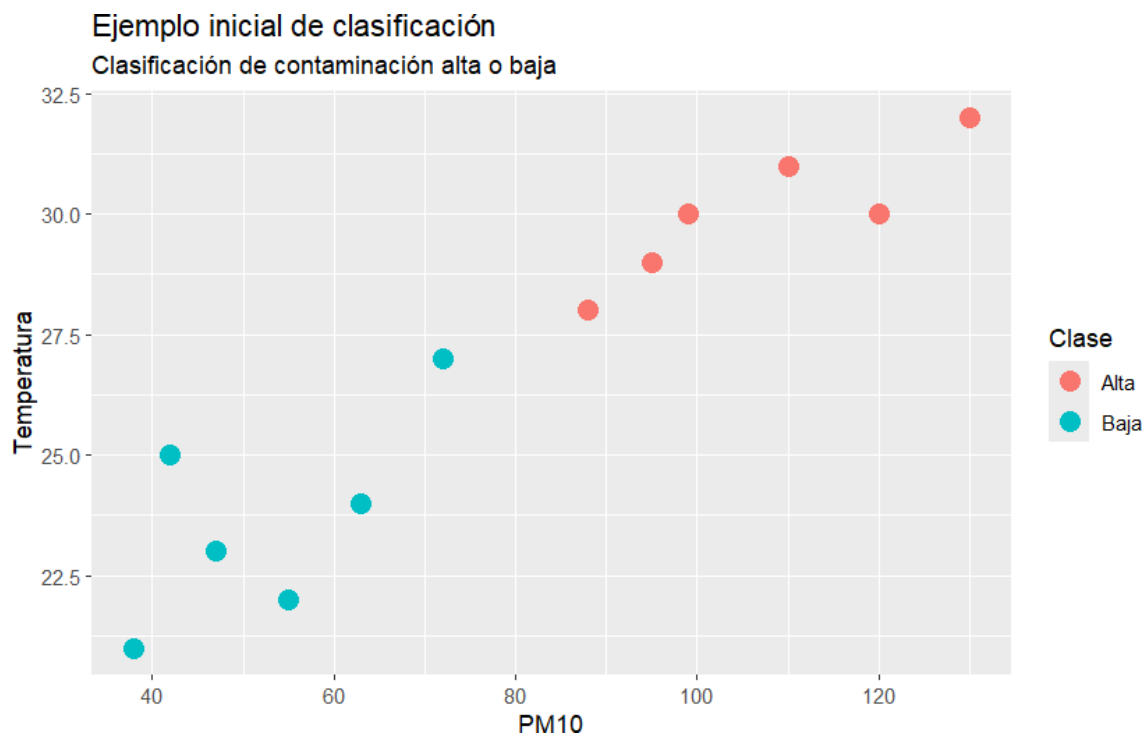
1.12. Flujo general de un proyecto de aprendizaje automático

1. Plantear el problema
2. Conseguir los datos
3. Entender las variables
4. Limpiar los datos
5. Explorar los datos
6. Dividir en entrenamiento y prueba
7. Entrenar el modelo
8. Evaluar el modelo
9. Interpretar los resultados
10. Comunicar hallazgos

1.13. Primer ejemplo visual en R

```
library(ggplot2)

ggplot(datos, aes(x = pm10, y = temperatura, color = clase)) +
  geom_point(size = 4) +
  labs(
    title = "Ejemplo inicial de clasificación",
    subtitle = "Clasificación de contaminación alta o baja",
    x = "PM10",
    y = "Temperatura",
    color = "Clase"
  )
```



Explicación del código.

La función `ggplot()` construye una gráfica de dispersión. En el eje horizontal se coloca `pm10`, en el eje vertical `temperatura` y el color representa la clase.

Interpretación del resultado.

La gráfica muestra que los días de contaminación alta tienden a ubicarse en la zona de mayor PM10 y mayor temperatura. Esta separación visual ilustra la idea de clasificación.

1.14. División entre entrenamiento y prueba

```
set.seed(123)

n <- nrow(datos)
```

```
indices_entrenamiento <- sample(1:n, size = round(0.7 * n))
```

```
entrenamiento <- datos[indices_entrenamiento, ]
```

```
prueba <- datos[-indices_entrenamiento, ]
```

```
entrenamiento
```

	dia	pm10	temperatura	humedad	clase
3	3	55	22	60	Baja
12	12	99	30	34	Alta
10	10	130	32	28	Alta
2	2	88	28	35	Alta
6	6	95	29	33	Alta
11	11	47	23	62	Baja
5	5	63	24	55	Baja
4	4	120	30	30	Alta

```
prueba
```

	dia	pm10	temperatura	humedad	clase
1	1	42	25	40	Baja
7	7	38	21	65	Baja
8	8	110	31	29	Alta
9	9	72	27	42	Baja

Explicación del código.

`set.seed()` permite reproducir la misma selección aleatoria. Después se toma aproximadamente 70% de las filas para entrenamiento y 30% para prueba.

Interpretación del resultado.

El conjunto de entrenamiento se usa para aprender patrones. El conjunto de prueba permite revisar si el modelo funcionaría con datos no vistos.

1.15. Primer clasificador sencillo basado en una regla

```
datos$prediccion_regla <- ifelse(datos$pm10 > 75, "Alta", "Baja")
```

```
datos[, c("pm10", "clase", "prediccion_regla")]
```

	pm10	clase	prediccion_regla
1	42	Baja	Baja
2	88	Alta	Alta
3	55	Baja	Baja
4	120	Alta	Alta
5	63	Baja	Baja
6	95	Alta	Alta
7	38	Baja	Baja
8	110	Alta	Alta
9	72	Baja	Baja
10	130	Alta	Alta

11	47	Baja	Baja
12	99	Alta	Alta

Explicación del código.

Se crea una nueva columna llamada `prediccion_regla`. La regla clasifica como “Alta” cuando PM10 es mayor que 75.

Interpretación del resultado.

La tabla permite comparar la clase real con una predicción generada por una regla simple.

```
tabla_confusion <- table(
  Real = datos$clase,
  Predicho = datos$prediccion_regla
)

tabla_confusion
```

		Predicho	
Real		Alta	Baja
Alta	6	0	
Baja	0	6	

Explicación del código.

La función `table()` cruza la clase real con la clase predicha.

Interpretación del resultado.

La diagonal principal contiene los aciertos. Los valores fuera de la diagonal serían errores de clasificación.

```
exactitud <- mean(datos$clase == datos$prediccion_regla)
exactitud
```

```
[1] 1
```

Explicación del código.

Se compara cada predicción con la clase real y se calcula la proporción de aciertos.

Interpretación del resultado.

La exactitud indica qué proporción de observaciones fueron clasificadas correctamente. En datos reales los resultados suelen ser más complejos.

1.16. Conclusión

El aprendizaje automático permite construir modelos que aprenden patrones a partir de datos. En este capítulo vimos la diferencia entre reglas manuales y aprendizaje a partir de ejemplos.

Capítulo 2

Preparación de datos reales

2.1. Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de leer una base real, seleccionar variables, construir una variable respuesta y guardar una base preparada para modelado.

2.2. Introducción

En la práctica profesional, los datos reales casi nunca vienen preparados. Antes de aplicar un algoritmo necesitamos revisar, limpiar, transformar y documentar los datos.

2.3. Datos crudos y datos preparados

Llamaremos **datos crudos** a la base tal como se obtiene de la fuente original. Llamaremos **datos preparados** a la base que ya fue revisada, transformada y organizada para un propósito específico.

2.4. Representación matemática de una base preparada

Una base de datos para aprendizaje automático puede representarse como una matriz de variables predictoras X y una variable respuesta Y .

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

En nuestro caso, Y representa la variable `accidente_con_victimas`.

$$Y_i = \begin{cases} 1, & \text{si el accidente tiene víctimas} \\ 0, & \text{si el accidente es de solo daños} \end{cases}$$

2.5. Algoritmo general de preparación de datos

Algoritmo: Preparación de datos para aprendizaje automático

Entrada:

- Base de datos cruda
- Diccionario de variables
- Pregunta de predicción

Proceso:

1. Leer la base de datos.
2. Revisar dimensiones, nombres y tipos de variables.
3. Seleccionar variables relevantes.
4. Identificar la variable respuesta.
5. Transformar variables cuando sea necesario.
6. Revisar datos faltantes.
7. Guardar una base preparada.

Salida:

- Base limpia y lista para análisis exploratorio y modelado

2.6. Conjunto de datos de trabajo: ATUS

Trabajaremos con **Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas**, conocido como ATUS, publicado por INEGI.

La pregunta práctica será:

¿Podemos clasificar si un accidente tendrá víctimas o será solo de daños materiales?

2.7. Crear la carpeta de datos

Dentro de la carpeta del libro crea una carpeta llamada `datos`. El archivo de datos debe llamarse `datos/atus_2024.csv`.

2.8. Cargar paquetes

```
library(readr)
library(dplyr)
library(ggplot2)
library(stringr)
```

Explicación del código.

readr permite leer archivos CSV, dplyr permite transformar tablas, ggplot2 permite graficar y stringr facilita trabajar con texto.

2.9. Leer el archivo CSV

```
ruta_atus <- "datos/atus_2024.csv"
file.exists(ruta_atus)

[1] TRUE

if (file.exists(ruta_atus)) {
  atus <- read_csv(
    file = ruta_atus,
    show_col_types = FALSE,
    locale = locale(encoding = "UTF-8")
  )
  print("Archivo leído correctamente.")
} else {
  atus <- NULL
  print("El archivo atus_2024.csv todavía no está disponible en la carpeta datos.")
}
```

```
[1] "Archivo leído correctamente."
```

Interpretación del resultado.

Si aparece el mensaje de carga correcta, se creó el objeto atus y podemos continuar.

2.10. Primer vistazo a los datos

```
if (!is.null(atus)) {
  data.frame(filas = nrow(atus), columnas = ncol(atus))
}
```

```
      filas columnas
1 390627         45
```

```
if (!is.null(atus)) {
  head(atus)
}
```

```
# A tibble: 6 x 45
  COBERTURA ID_ENTIDAD ID_MUNICIPIO ANIO MES ID_HORA ID_MINUTO ID_DIA
  <chr>      <chr>      <chr>      <dbl> <chr> <chr>      <chr>      <chr>
1 Municipal 01        001        2024 01    02        25        01
2 Municipal 01        001        2024 01    03        40        01
3 Municipal 01        001        2024 01    04        37        01
4 Municipal 01        001        2024 01    05        00        01
5 Municipal 01        001        2024 01    06        45        01
```

```
6 Municipal 01          001          2024 01    07      30      01
# i 37 more variables: DIASEMANA <chr>, URBANA <chr>, SUBURBANA <chr>,
#   TIPACCID <chr>, AUTOMOVIL <dbl>, CAMPASAJ <dbl>, MICROBUS <dbl>,
#   PASCAMION <dbl>, OMNIBUS <dbl>, TRANVIA <dbl>, CAMIONETA <dbl>,
#   CAMION <dbl>, TRACTOR <dbl>, FERROCARRI <dbl>, MOTOCICLET <dbl>,
#   BICICLETA <dbl>, OTROVEHIC <dbl>, CAUSAACCI <chr>, CAPAROD <chr>,
#   SEXO <chr>, ALIENTO <chr>, CINTURON <chr>, ID_EDAD <dbl>, CONDMUERTO <dbl>,
#   CONDHERIDO <dbl>, PASAMUERTO <dbl>, PASAHERIDO <dbl>, PEATMUERTO <dbl>, ...
```

```
if (!is.null(atus)) {
  names(atus)
}
```

[1]	"COBERTURA"	"ID_ENTIDAD"	"ID_MUNICIPIO"	"ANIO"	"MES"
[6]	"ID_HORA"	"ID_MINUTO"	"ID_DIA"	"DIASEMANA"	"URBANA"
[11]	"SUBURBANA"	"TIPACCID"	"AUTOMOVIL"	"CAMPASAJ"	"MICROBUS"
[16]	"PASCAMION"	"OMNIBUS"	"TRANVIA"	"CAMIONETA"	"CAMION"
[21]	"TRACTOR"	"FERROCARRI"	"MOTOCICLET"	"BICICLETA"	"OTROVEHIC"
[26]	"CAUSAACCI"	"CAPAROD"	"SEXO"	"ALIENTO"	"CINTURON"
[31]	"ID_EDAD"	"CONDMUERTO"	"CONDHERIDO"	"PASAMUERTO"	"PASAHERIDO"
[36]	"PEATMUERTO"	"PEATHERIDO"	"CICLMUERTO"	"CICLHERIDO"	"OTROMUERTO"
[41]	"OTROHERIDO"	"NEMUERTO"	"NEHERIDO"	"CLASACC"	"ESTATUS"

Interpretación del resultado.

Este paso permite confirmar que la base fue cargada correctamente y que contiene las variables necesarias.

2.11. Estandarizar nombres de variables

```
if (!is.null(atus)) {
  names(atus) <- toupper(names(atus))
  names(atus)
}
```

[1]	"COBERTURA"	"ID_ENTIDAD"	"ID_MUNICIPIO"	"ANIO"	"MES"
[6]	"ID_HORA"	"ID_MINUTO"	"ID_DIA"	"DIASEMANA"	"URBANA"
[11]	"SUBURBANA"	"TIPACCID"	"AUTOMOVIL"	"CAMPASAJ"	"MICROBUS"
[16]	"PASCAMION"	"OMNIBUS"	"TRANVIA"	"CAMIONETA"	"CAMION"
[21]	"TRACTOR"	"FERROCARRI"	"MOTOCICLET"	"BICICLETA"	"OTROVEHIC"
[26]	"CAUSAACCI"	"CAPAROD"	"SEXO"	"ALIENTO"	"CINTURON"
[31]	"ID_EDAD"	"CONDMUERTO"	"CONDHERIDO"	"PASAMUERTO"	"PASAHERIDO"
[36]	"PEATMUERTO"	"PEATHERIDO"	"CICLMUERTO"	"CICLHERIDO"	"OTROMUERTO"
[41]	"OTROHERIDO"	"NEMUERTO"	"NEHERIDO"	"CLASACC"	"ESTATUS"

Explicación del código.

`toupper()` convierte los nombres de las columnas a mayúsculas. Esto ayuda a evitar errores por diferencias entre mayúsculas y minúsculas.

2.12. Variables de interés

```
variables_interes <- c(
  "ANIO", "MES", "ID_ENTIDAD", "ID_MUNICIPIO",
  "ID_HORA", "ID_DIA", "DIASEMANA",
  "TIPACCID", "CAUSAACCI", "CLASACC"
)

if (!is.null(atus)) {
  variables_existentes <- intersect(variables_interes, names(atus))
  variables_existentes
}
```

```
[1] "ANIO"          "MES"          "ID_ENTIDAD"   "ID_MUNICIPIO" "ID_HORA"
[6] "ID_DIA"        "DIASEMANA"    "TIPACCID"     "CAUSAACCI"    "CLASACC"
```

```
if (!is.null(atus)) {
  atus_base <- atus |>
    select(all_of(variables_existentes))

  head(atus_base)
}
```

```
# A tibble: 6 x 10
  ANIO MES ID_ENTIDAD ID_MUNICIPIO ID_HORA ID_DIA DIASEMANA TIPACCID
<dbl> <chr> <chr>      <chr>      <chr>  <chr>  <chr>      <chr>
1  2024 01 01        001        02    01    lunes    Colisión con veh~
2  2024 01 01        001        03    01    lunes    Colisión con obj~
3  2024 01 01        001        04    01    lunes    Colisión con pea~
4  2024 01 01        001        05    01    lunes    Colisión con obj~
5  2024 01 01        001        06    01    lunes    Colisión con obj~
6  2024 01 01        001        07    01    lunes    Colisión con veh~
# i 2 more variables: CAUSAACCI <chr>, CLASACC <chr>
```

Interpretación del resultado.

La base queda más manejable y enfocada en el problema de clasificación.

2.13. Diccionario inicial de variables

Variable	Descripción didáctica
ANIO	Año del accidente
MES	Mes del accidente
ID_ENTIDAD	Clave de la entidad federativa
ID_MUNICIPIO	Clave del municipio
ID_HORA	Hora del accidente
ID_DIA	Día del mes
DIASEMANA	Día de la semana
TIPACCID	Tipo de accidente

Variable	Descripción didáctica
CAUSAACCI	Causa presunta del accidente
CLASACC	Clase del accidente

2.14. Revisión de la variable CLASACC

```
if (exists("atus_base") && "CLASACC" %in% names(atus_base)) {
  table(atus_base$CLASACC, useNA = "ifany")
}
```

Fatal	No fatal	Sólo daños
4188	63920	322519

Interpretación del resultado.

CLASACC distingue accidentes fatales, no fatales y de solo daños. A partir de esta variable construiremos una respuesta binaria.

2.15. Crear una variable respuesta

```
if (exists("atus_base") && "CLASACC" %in% names(atus_base)) {
  atus_modelo <- atus_base |>
  mutate(
    CLASACC_TXT = as.character(CLASACC),
    CLASACC_TXT = str_to_lower(CLASACC_TXT),
    accidente_con_victimas = case_when(
      str_detect(CLASACC_TXT, "sólo daños") ~ "Solo daños",
      str_detect(CLASACC_TXT, "solo daños") ~ "Solo daños",
      str_detect(CLASACC_TXT, "daños") ~ "Solo daños",
      str_detect(CLASACC_TXT, "no fatal") ~ "Con víctimas",
      str_detect(CLASACC_TXT, "fatal") ~ "Con víctimas",
      CLASACC_TXT %in% c("1", "2") ~ "Con víctimas",
      CLASACC_TXT %in% c("3") ~ "Solo daños",
      TRUE ~ NA_character_
    )
  )
}

if (exists("atus_modelo")) {
  table(atus_modelo$accidente_con_victimas, useNA = "ifany")
}
```

Con víctimas	Solo daños
68108	322519

Interpretación del resultado.

La tabla muestra cuántos registros quedaron en cada clase. Esta será la variable respuesta del problema de clasificación.

2.16. Eliminar registros sin variable respuesta

```
if (exists("atus_modelo")) {
  atus_modelo <- atus_modelo |>
    filter(!is.na(accidente_con_victimas))

  table(atus_modelo$accidente_con_victimas)
}
```

Con víctimas	Solo daños
68108	322519

2.17. Convertir variables categóricas

```
if (exists("atus_modelo")) {
  atus_modelo <- atus_modelo |>
    mutate(
      ID_ENTIDAD = as.factor(ID_ENTIDAD),
      ID_MUNICIPIO = as.factor(ID_MUNICIPIO),
      MES = as.factor(MES),
      DIASEMANA = as.factor(DIASEMANA),
      TIPACCID = as.factor(TIPACCID),
      CAUSAACCI = as.factor(CAUSAACCI),
      accidente_con_victimas = as.factor(accidente_con_victimas)
    )

  str(atus_modelo)
}
```

```
tibble [390,627 x 12] (S3: tbl_df/tbl/data.frame)
 $ ANIO                : num [1:390627] 2024 2024 2024 2024 2024 ...
 $ MES                 : Factor w/ 12 levels "01","02","03",...: 1 1 1 1 1 1 1 1 1 1 ...
 $ ID_ENTIDAD          : Factor w/ 32 levels "01","02","03",...: 1 1 1 1 1 1 1 1 1 1 ...
 $ ID_MUNICIPIO        : Factor w/ 570 levels "001","002","003",...: 1 1 1 1 1 1 1 1 1 1 ...
 $ ID_HORA             : chr [1:390627] "02" "03" "04" "05" ...
 $ ID_DIA              : chr [1:390627] "01" "01" "01" "01" ...
 $ DIASEMANA           : Factor w/ 7 levels "Domingo","Jueves",...: 3 3 3 3 3 3 3 3 3 3 ...
 $ TIPACCID            : Factor w/ 13 levels "Caída de pasajero",...: 9 7 8 7 7 9 7 9 7 9 ...
 $ CAUSAACCI           : Factor w/ 6 levels "Certificado cero",...: 2 2 2 2 2 2 2 2 2 2 ...
 $ CLASACC             : chr [1:390627] "Sólo daños" "Sólo daños" "No fatal" "Sólo daños" ...
 $ CLASACC_TXT         : chr [1:390627] "sólo daños" "sólo daños" "no fatal" "sólo daños" ...
 $ accidente_con_victimas: Factor w/ 2 levels "Con víctimas",...: 2 2 1 2 2 1 1 2 2 2 ...
```

Interpretación del resultado.

Variables como entidad o municipio son claves de identificación; no deben interpretarse como cantidades numéricas.

2.18. Revisión de datos faltantes

```
if (exists("atus_modelo")) {
  colSums(is.na(atus_modelo))
}
```

ANIO	MES	ID_ENTIDAD
0	0	0
ID_MUNICIPIO	ID_HORA	ID_DIA
0	0	0
DIASEMANA	TIPACCID	CAUSAACCI
0	0	0
CLASACC	CLASACC_TXT	accidente_con_victimas
0	0	0

```
if (exists("atus_modelo")) {
  round(100 * colSums(is.na(atus_modelo)) / nrow(atus_modelo), 2)
}
```

ANIO	MES	ID_ENTIDAD
0	0	0
ID_MUNICIPIO	ID_HORA	ID_DIA
0	0	0
DIASEMANA	TIPACCID	CAUSAACCI
0	0	0
CLASACC	CLASACC_TXT	accidente_con_victimas
0	0	0

2.19. Selección final de variables para el primer modelo

```
if (exists("atus_modelo")) {
  atus_ml <- atus_modelo |>
  select(
    accidente_con_victimas,
    MES,
    ID_HORA,
    DIASEMANA,
    TIPACCID,
    CAUSAACCI
  )

  head(atus_ml)
}
```

```
# A tibble: 6 x 6
  accidente_con_victimas MES ID_HORA DIASEMANA TIPACCID CAUSAACCI
  <fct> <fct> <chr> <fct> <fct> <fct>
1 Solo daños 01 02 lunes Colisión con vehícul~ Conductor
2 Solo daños 01 03 lunes Colisión con objeto ~ Conductor
3 Con víctimas 01 04 lunes Colisión con peatón ~ Conductor
4 Solo daños 01 05 lunes Colisión con objeto ~ Conductor
5 Solo daños 01 06 lunes Colisión con objeto ~ Conductor
6 Con víctimas 01 07 lunes Colisión con vehícul~ Conductor
```

```
if (exists("atus_ml")) {
  atus_ml <- na.omit(atus_ml)
  data.frame(filas = nrow(atus_ml), columnas = ncol(atus_ml))
}
```

```
      filas columnas
1 390627          6
```

2.20. Guardar la base preparada

```
if (exists("atus_ml")) {
  write_csv(atus_ml, "datos/atus_ml_preparado.csv")
}
```

Interpretación del resultado.

El archivo `atus_ml_preparado.csv` evita repetir todo el proceso de limpieza en los siguientes capítulos.

2.21. Resumen del capítulo

En este capítulo leímos una base real, seleccionamos variables, construimos una variable respuesta y guardamos una base preparada para aprendizaje automático.

Capítulo 3

Análisis exploratorio de datos

3.1. Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de cargar una base preparada, revisar su estructura, calcular frecuencias, construir gráficas e interpretar patrones iniciales.

3.2. Introducción

El **análisis exploratorio de datos** permite revisar la estructura de la base, detectar patrones iniciales e identificar posibles problemas antes de construir modelos.

3.3. Fundamento matemático del análisis exploratorio

Si una variable categórica tiene categorías $c_1, c_2, \dots, c_k$, la frecuencia absoluta de una categoría c_j se representa como:

$$n_j = \sum_{i=1}^n I(y_i = c_j)$$

La proporción de la categoría c_j es:

$$p_j = \frac{n_j}{n}$$

3.4. Algoritmo general

Algoritmo: Análisis exploratorio de datos

Entrada:

- Base de datos preparada
- Variable respuesta
- Variables predictoras

Proceso:

1. Revisar dimensiones de la base.
2. Revisar nombres y tipos de variables.
3. Calcular frecuencias y proporciones.
4. Construir gráficas exploratorias.
5. Cruzar variables predictoras con la variable respuesta.
6. Identificar patrones y posibles problemas.

Salida:

- Tablas descriptivas
- Gráficas exploratorias
- Primeras hipótesis sobre los datos

3.5. Cargar paquetes y base

```
library(readr)
library(dplyr)
library(ggplot2)

ruta_atus_ml <- "datos/atus_ml_preparado.csv"
file.exists(ruta_atus_ml)
```

```
[1] TRUE
```

```
if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
} else {
  atus_ml <- NULL
  print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo 2.")
}
```

```
[1] "Base preparada cargada correctamente."
```

3.6. Preparar variables para el análisis exploratorio

```
if (!is.null(atus_ml)) {
  atus_ml <- atus_ml |>
  mutate(
    ID_HORA_NUM = as.numeric(ID_HORA),
    MES_NUM = as.numeric(MES),
    MES_FACTOR = factor(sprintf("%02d", MES_NUM), levels = sprintf("%02d", 1:12)),
    DIASEMANA = factor(
      DIASEMANA,
      levels = c("lunes", "Martes", "Miercoles", "Jueves", "Viernes", "Sabado", "Domingo")
    ),
    accidente_con_victimas = factor(
```

```

    accidente_con_victimas,
    levels = c("Con víctimas", "Solo daños")
  )
}

```

Explicación del código.

Se crean variables ordenadas para hora, mes y día de la semana. MES_FACTOR evita que la gráfica mensual genere una categoría NA.

3.7. Primer vistazo a la base

```

if (!is.null(atus_ml)) {
  data.frame(filas = nrow(atus_ml), columnas = ncol(atus_ml))
}

```

```

      filas columnas
1 390627          9

```

```

if (!is.null(atus_ml)) {
  head(atus_ml)
}

```

A tibble: 6 x 9

	accidente_con_victimas	MES	ID_HORA	DIASEMANA	TIPACCID	CAUSAACCI	ID_HORA_NUM
	<fct>	<chr>	<chr>	<fct>	<chr>	<chr>	<dbl>
1	Solo daños	01	02	lunes	Colisión~	Conductor	2
2	Solo daños	01	03	lunes	Colisión~	Conductor	3
3	Con víctimas	01	04	lunes	Colisión~	Conductor	4
4	Solo daños	01	05	lunes	Colisión~	Conductor	5
5	Solo daños	01	06	lunes	Colisión~	Conductor	6
6	Con víctimas	01	07	lunes	Colisión~	Conductor	7

i 2 more variables: MES_NUM <dbl>, MES_FACTOR <fct>

```

if (!is.null(atus_ml)) {
  str(atus_ml)
}

```

tibble [390,627 x 9] (S3: tbl_df/tbl/data.frame)

```

$ accidente_con_victimas: Factor w/ 2 levels "Con víctimas",...: 2 2 1 2 2 1 1 2 2 2 ...
$ MES                   : chr [1:390627] "01" "01" "01" "01" ...
$ ID_HORA               : chr [1:390627] "02" "03" "04" "05" ...
$ DIASEMANA             : Factor w/ 7 levels "lunes","Martes",...: 1 1 1 1 1 1 1 1 1 1 ...
$ TIPACCID              : chr [1:390627] "Colisión con vehículo automotor" "Colisión con objet...
$ CAUSAACCI             : chr [1:390627] "Conductor" "Conductor" "Conductor" "Conductor" ...
$ ID_HORA_NUM           : num [1:390627] 2 3 4 5 6 7 12 13 18 20 ...
$ MES_NUM               : num [1:390627] 1 1 1 1 1 1 1 1 1 1 ...
$ MES_FACTOR            : Factor w/ 12 levels "01","02","03",...: 1 1 1 1 1 1 1 1 1 1 ...

```

```
if (!is.null(atus_ml)) {
  tabla_clase <- table(atus_ml$accidente_con_victimas)
  tabla_clase
}
```

Con víctimas	Solo daños
68108	322519

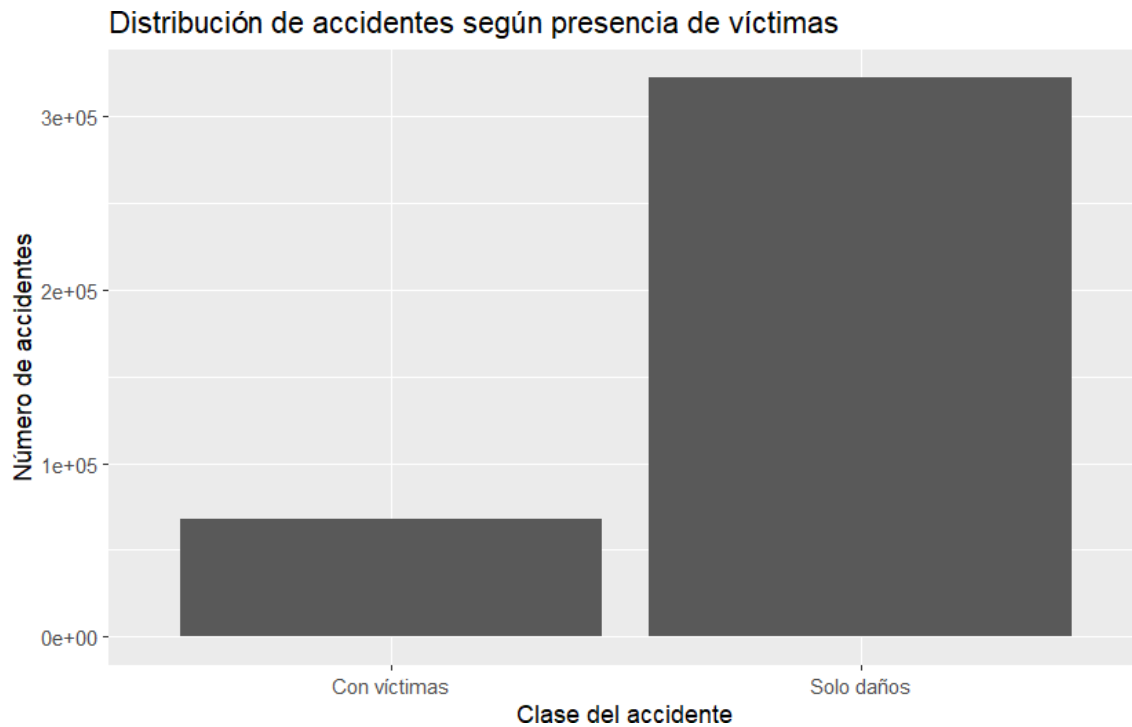
```
if (!is.null(atus_ml)) {
  round(100 * prop.table(tabla_clase), 2)
}
```

Con víctimas	Solo daños
17.44	82.56

Interpretación del resultado.

La base está desbalanceada: hay muchos más accidentes de solo daños que accidentes con víctimas.

```
if (!is.null(atus_ml)) {
  ggplot(atus_ml, aes(x = accidente_con_victimas)) +
    geom_bar() +
    labs(
      title = "Distribución de accidentes según presencia de víctimas",
      x = "Clase del accidente",
      y = "Número de accidentes"
    )
}
```



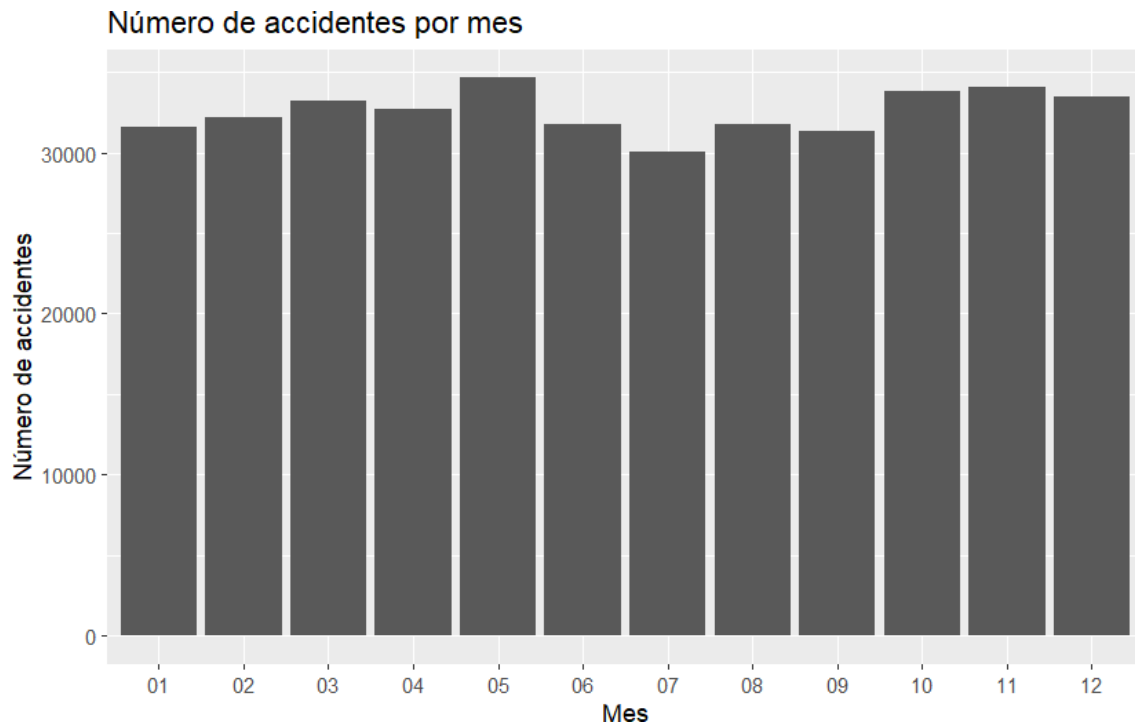
3.9. Accidentes por mes

```
if (!is.null(atus_ml)) {
  accidentes_mes <- atus_ml |>
    count(MES_FACTOR, name = "n") |>
    arrange(MES_FACTOR)

  accidentes_mes
}
```

```
# A tibble: 12 x 2
  MES_FACTOR     n
  <fct>       <int>
1 01         31600
2 02         32208
3 03         33237
4 04         32666
5 05         34665
6 06         31737
7 07         30062
8 08         31800
9 09         31351
10 10         33771
11 11         34047
12 12         33483
```

```
if (!is.null(atus_ml)) {
  ggplot(accidentes_mes, aes(x = MES_FACTOR, y = n)) +
    geom_col() +
    labs(
      title = "Número de accidentes por mes",
      x = "Mes",
      y = "Número de accidentes"
    )
}
```



Interpretación del resultado.

La gráfica muestra variaciones mensuales sin crear una categoría NA.

3.10. Accidentes por hora del día

```
if (!is.null(atus_ml)) {
  accidentes_hora <- atus_ml |>
    count(ID_HORA_NUM, name = "n") |>
    arrange(ID_HORA_NUM)

  accidentes_hora
}
```

```
# A tibble: 24 x 2
  ID_HORA_NUM      n
  <dbl> <int>
1         0 9143
```

```

2          1 7400
3          2 6594
4          3 5694
5          4 4880
6          5 4986
7          6 8034
8          7 15159
9          8 20666
10         9 19486

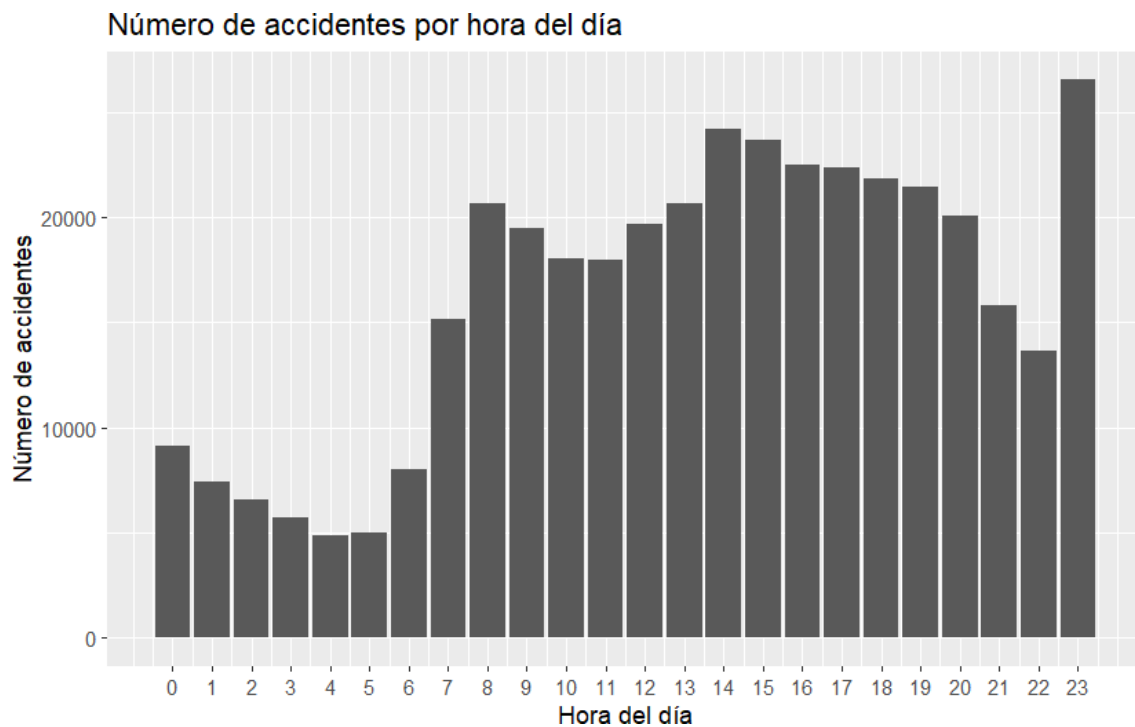
```

```
# i 14 more rows
```

```

if (!is.null(atus_ml)) {
  ggplot(accidentes_hora, aes(x = ID_HORA_NUM, y = n)) +
    geom_col() +
    scale_x_continuous(breaks = 0:23) +
    labs(
      title = "Número de accidentes por hora del día",
      x = "Hora del día",
      y = "Número de accidentes"
    )
}

```



3.11. Accidentes por día de la semana

```

if (!is.null(atus_ml)) {
  accidentes_dia <- atus_ml |>
  count(DIASEMANA, name = "n") |>

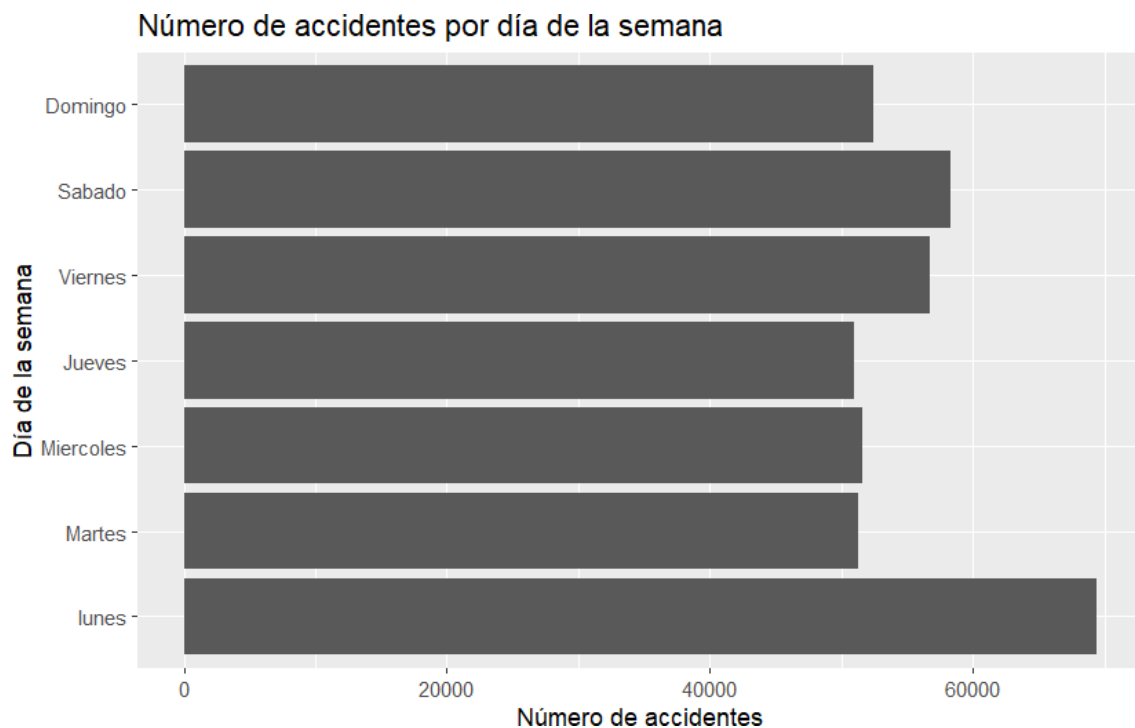
```

```
    arrange(DIASEMANA)

    accidentes_dia
  }
```

```
# A tibble: 7 x 2
  DIASEMANA      n
  <fct>      <int>
1 lunes      69412
2 Martes     51215
3 Miercoles  51580
4 Jueves     50954
5 Viernes    56708
6 Sabado     58321
7 Domingo    52437
```

```
if (!is.null(atus_ml)) {
  ggplot(accidentes_dia, aes(x = DIASEMANA, y = n)) +
    geom_col() +
    coord_flip() +
    labs(
      title = "Número de accidentes por día de la semana",
      x = "Día de la semana",
      y = "Número de accidentes"
    )
}
```



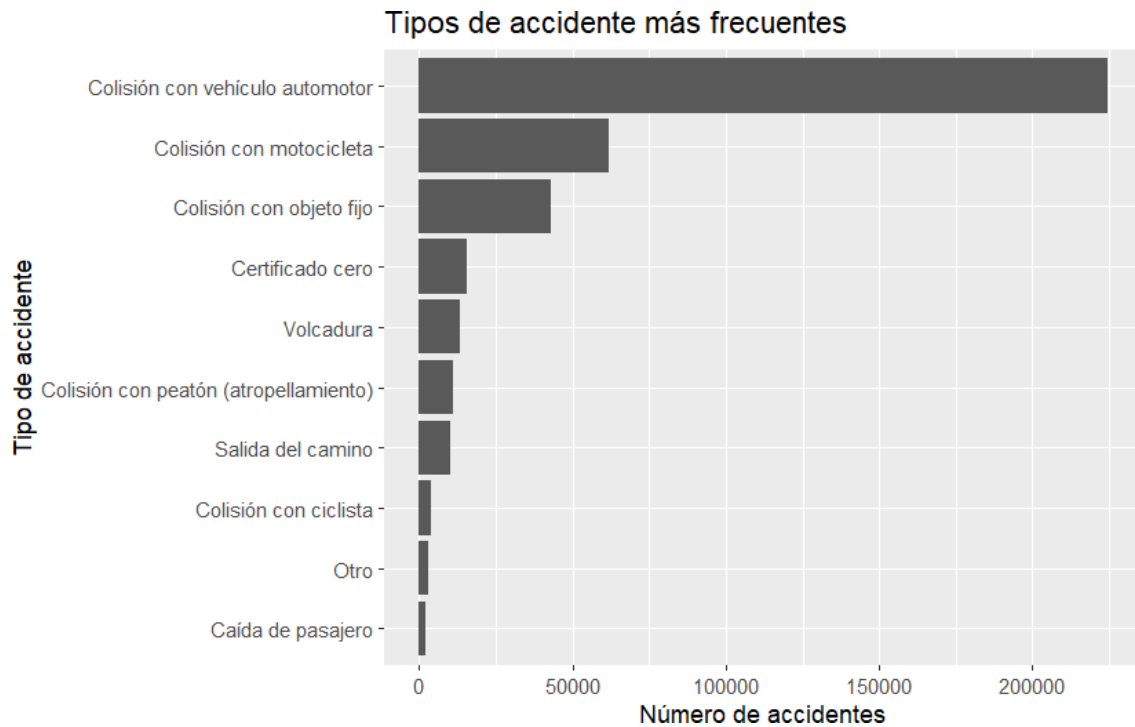
```
accidentes_tipo |> head(10)
}
```

3.12. Accidentes por tipo de accidente

```
# A tibble: 10 x 2
  TIPACCID          n
  <chr>          <int>
1 Colisión con vehículo automotor 224720
2 Colisión con motocicleta      61869
3 Colisión con objeto fijo      43119
4 Certificado cero             15678
5 Volcadura                    13371
6 Colisión con peatón (atropellamiento) 11052
7 Salida del camino            10067
8 Colisión con ciclista         4033
9 Otro                         2934
10 Caída de pasajero           2153

if (!is.null(atus_ml)) {
  accidentes_tipo_top <- accidentes_tipo |>
    slice_max(n, n = 10)

  ggplot(accidentes_tipo_top, aes(x = reorder(TIPACCID, n), y = n)) +
    geom_col() +
    coord_flip() +
    labs(
      title = "Tipos de accidente más frecuentes",
      x = "Tipo de accidente",
      y = "Número de accidentes"
    )
}
```



3.13. Accidentes por causa presunta

```
if (!is.null(atus_ml)) {
  accidentes_causa <- atus_ml |>
    count(CAUSAACCI, name = "n") |>
    arrange(desc(n))

  accidentes_causa
}
```

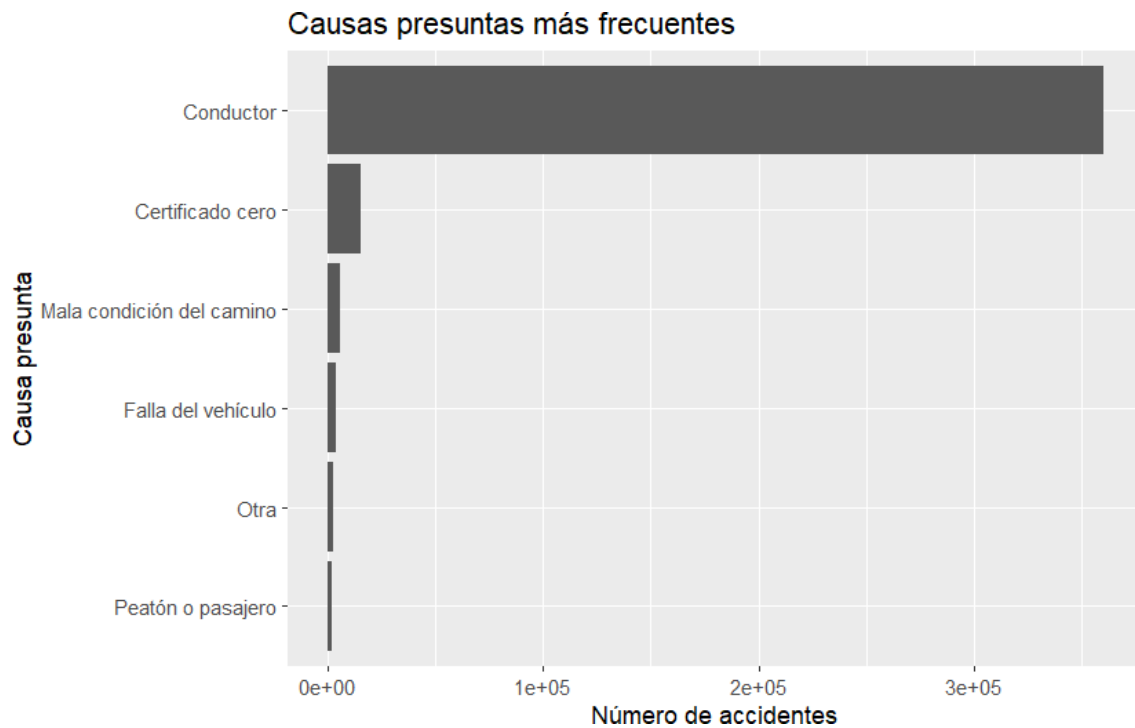
```
# A tibble: 6 x 2
  CAUSAACCI      n
  <chr>        <int>
1 Conductor    360051
2 Certificado cero 15678
3 Mala condición del camino 5991
4 Falla del vehículo 3978
5 Otra         2639
6 Peatón o pasajero 2290
```

```
if (!is.null(atus_ml)) {
  ggplot(accidentes_causa, aes(x = reorder(CAUSAACCI, n), y = n)) +
    geom_col() +
    coord_flip() +
    labs(
      title = "Causas presuntas más frecuentes",
      x = "Causa presunta",
```

```

    y = "Número de accidentes"
  )
}

```



3.14. Relación entre tipo de accidente y presencia de víctimas

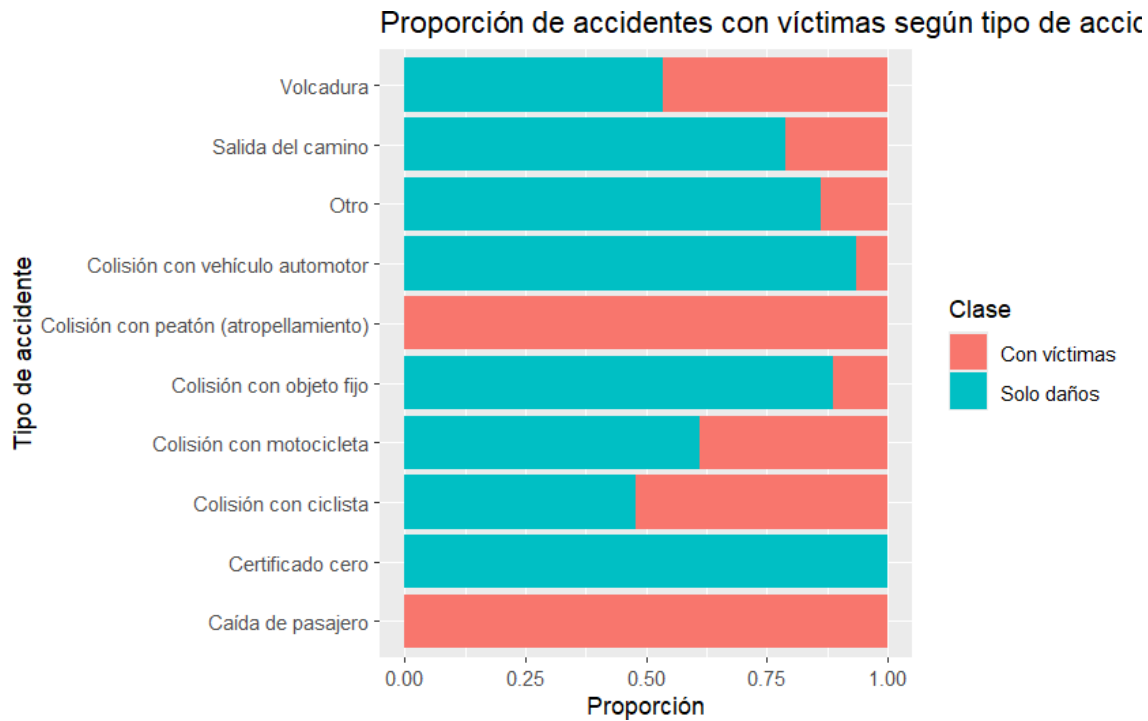
```

if (!is.null(atus_ml)) {
  tipos_top <- atus_ml |>
    count(TIPACCID) |>
    slice_max(n, n = 10) |>
    pull(TIPACCID)

  atus_tipo_top <- atus_ml |>
    filter(TIPACCID %in% tipos_top)

  ggplot(atus_tipo_top, aes(x = TIPACCID, fill = accidente_con_victimas)) +
    geom_bar(position = "fill") +
    coord_flip() +
    labs(
      title = "Proporción de accidentes con víctimas según tipo de accidente",
      x = "Tipo de accidente",
      y = "Proporción",
      fill = "Clase"
    )
}

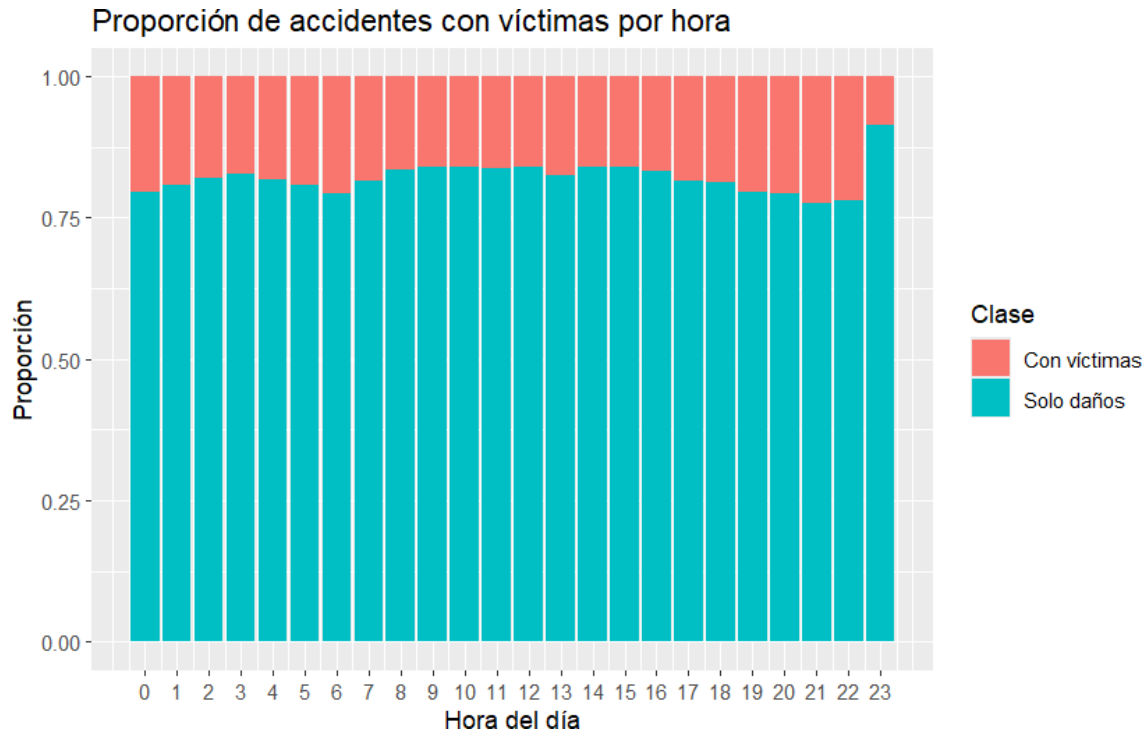
```



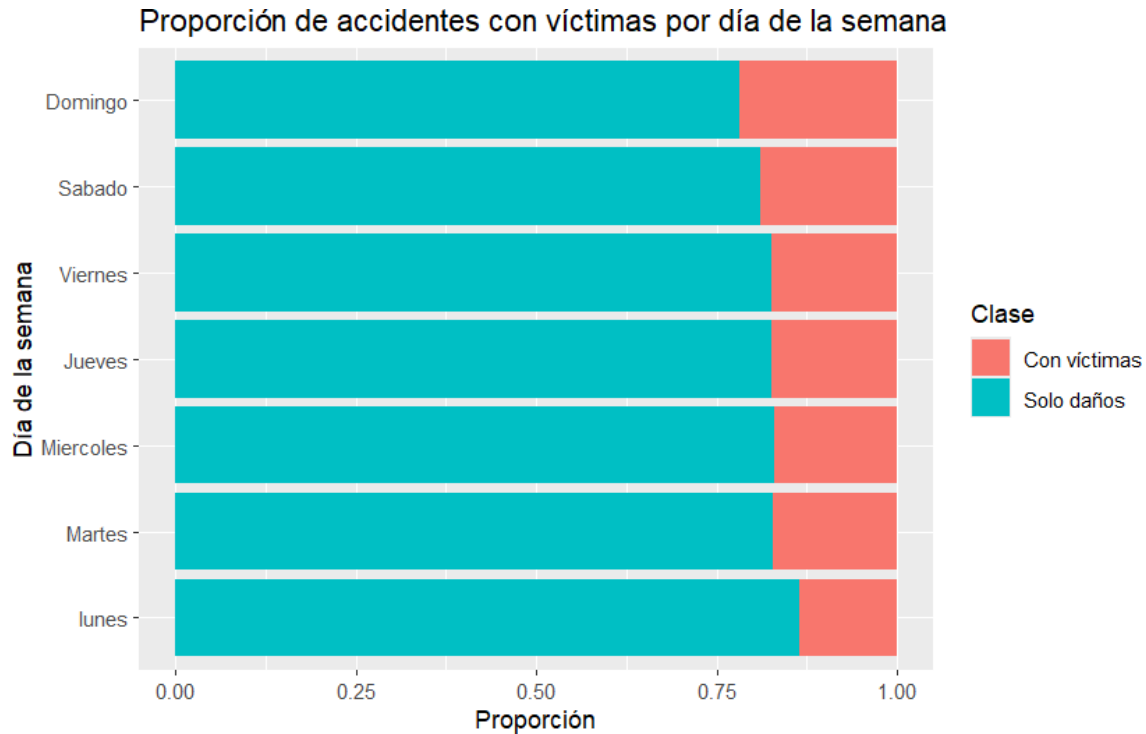
Interpretación del resultado.

Esta gráfica compara proporciones, no cantidades absolutas. Es útil para detectar tipos de accidente con mayor proporción relativa de víctimas.

```
if (!is.null(atus_ml)) {
  ggplot(atus_ml, aes(x = ID_HORA_NUM, fill = accidente_con_victimas)) +
    geom_bar(position = "fill") +
    scale_x_continuous(breaks = 0:23) +
    labs(
      title = "Proporción de accidentes con víctimas por hora",
      x = "Hora del día",
      y = "Proporción",
      fill = "Clase"
    )
}
```



```
if (!is.null(atus_ml)) {  
  ggplot(atus_ml, aes(x = DIASEMANA, fill = accidente_con_victimas)) +  
    geom_bar(position = "fill") +  
    coord_flip() +  
    labs(  
      title = "Proporción de accidentes con víctimas por día de la semana",  
      x = "Día de la semana",  
      y = "Proporción",  
      fill = "Clase"  
    )  
}
```



3.17. Tabla resumen por clase

```
if (!is.null(atus_ml)) {
  resumen_clase <- atus_ml |>
    group_by(accidente_con_victimas) |>
    summarise(
      total = n(),
      hora_promedio = mean(ID_HORA_NUM, na.rm = TRUE),
      hora_minima = min(ID_HORA_NUM, na.rm = TRUE),
      hora_maxima = max(ID_HORA_NUM, na.rm = TRUE),
      .groups = "drop"
    )

  resumen_clase
}
```

```
# A tibble: 2 x 5
  accidente_con_victimas total hora_promedio hora_minima hora_maxima
  <fct>                <int>      <dbl>         <dbl>         <dbl>
1 Con víctimas         68108      13.6           0             23
2 Solo daños           322519      13.7           0             23
```

3.18. Conclusión

En este capítulo aprendimos a cargar una base preparada, revisar su estructura, calcular frecuencias, construir gráficas y detectar patrones iniciales.

Capítulo 4

Regresión logística para clasificación

4.1. Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de ajustar una regresión logística, predecir probabilidades, construir una matriz de confusión e interpretar métricas de clasificación.

4.2. Introducción

El problema que estudiaremos es:

¿Podemos estimar la probabilidad de que un accidente de tránsito tenga víctimas?

La variable que queremos predecir es `accidente_con_victimas`.

4.3. La función logística

La regresión logística utiliza una función que convierte cualquier valor real en una probabilidad entre 0 y 1:

$$p = \frac{1}{1 + e^{-z}}$$

donde:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

4.4. Fundamento matemático

En clasificación binaria, la respuesta puede representarse como:

$$Y_i = \begin{cases} 1, & \text{si el accidente tiene víctimas} \\ 0, & \text{si el accidente es de solo daños} \end{cases}$$

La regresión logística estima $P(Y_i = 1 \mid X_i) = \hat{p}_i$ y clasifica con un punto de corte c .

Algoritmo: Regresión logística para clasificación binaria

Entrada:

- Matriz de variables predictoras X
- Variable respuesta binaria Y
- Punto de corte c

Proceso:

1. Codificar la variable respuesta como 0 y 1.
2. Estimar los coeficientes beta.
3. Calcular probabilidades con la función logística.
4. Convertir probabilidades en clases usando c .
5. Evaluar predicciones con matriz de confusión.

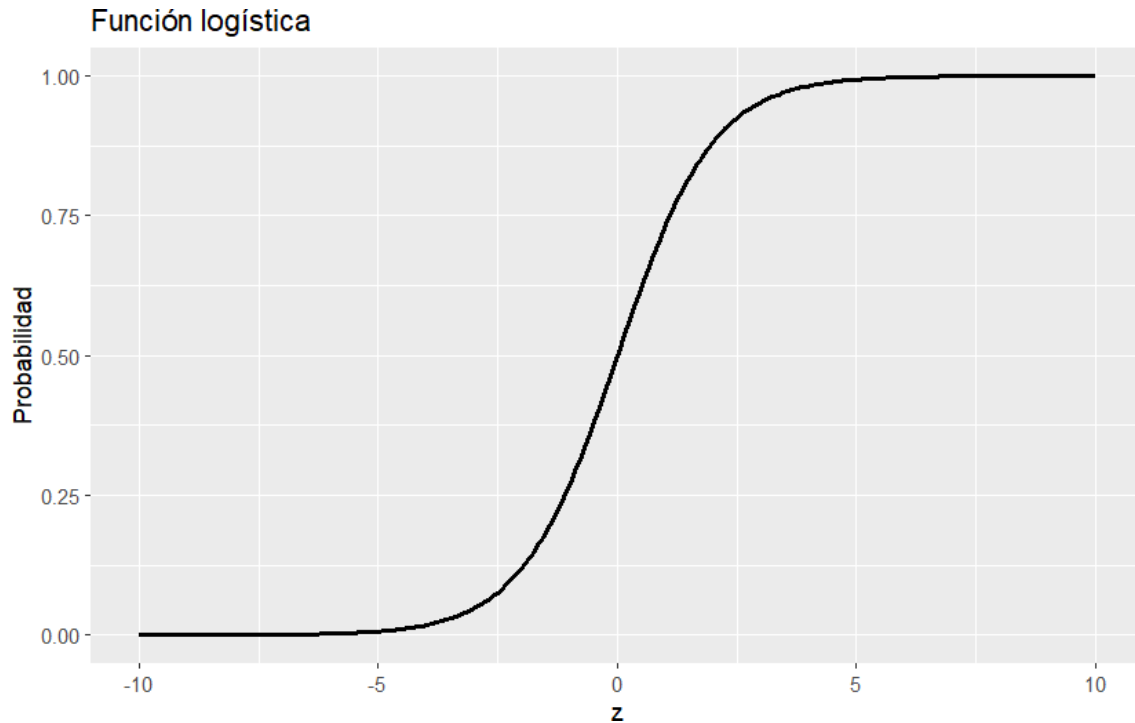
Salida:

- Probabilidades predichas
- Clases predichas
- Métricas de evaluación

```
z <- seq(-10, 10, length.out = 200)
p <- 1 / (1 + exp(-z))
datos_logistica <- data.frame(z = z, p = p)

library(ggplot2)

ggplot(datos_logistica, aes(x = z, y = p)) +
  geom_line(linewidth = 1) +
  labs(title = "Función logística", x = "z", y = "Probabilidad")
```



Interpretación del resultado.

La curva tiene forma de “S”. Para valores muy negativos de z , la probabilidad se acerca a 0. Para valores muy positivos, se acerca a 1.

4.7. Cargar paquetes y datos

```
library(readr)
library(dplyr)
library(ggplot2)

ruta_atus_ml <- "datos/atus_ml_preparado.csv"
file.exists(ruta_atus_ml)
```

```
[1] TRUE
```

```
if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
} else {
  atus_ml <- NULL
  print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo 2.")
}
```

```
[1] "Base preparada cargada correctamente."
```

4.8. Preparar variables para el modelo

```
if (!is.null(atus_ml)) {
  atus_modelo <- atus_ml |>
  mutate(
    victimas_binaria = ifelse(accidente_con_victimas == "Con víctimas", 1, 0),
    MES = as.factor(MES),
    DIASEMANA = as.factor(DIASEMANA),
    TIPACCID = as.factor(TIPACCID),
    CAUSAACCI = as.factor(CAUSAACCI),
    ID_HORA = as.numeric(ID_HORA),
    accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas", "Solo
  )

  table(atus_modelo$accidente_con_victimas)
}
```

Con víctimas	Solo daños
68108	322519

4.9. División en entrenamiento y prueba

```
if (exists("atus_modelo")) {
  set.seed(123)
  n <- nrow(atus_modelo)
  indices_entrenamiento <- sample(1:n, size = round(0.7 * n))
  entrenamiento <- atus_modelo[indices_entrenamiento, ]
  prueba <- atus_modelo[-indices_entrenamiento, ]

  data.frame(
    conjunto = c("Entrenamiento", "Prueba"),
    registros = c(nrow(entrenamiento), nrow(prueba)),
    porcentaje = c(
      round(100 * nrow(entrenamiento) / nrow(atus_modelo), 2),
      round(100 * nrow(prueba) / nrow(atus_modelo), 2)
    )
  )
}
```

	conjunto	registros	porcentaje
1	Entrenamiento	273439	70
2	Prueba	117188	30

4.10. Ajustar el modelo

```
if (exists("entrenamiento")) {
  modelo_logistico <- glm(
    victimas_binaria ~ MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
    data = entrenamiento,
    family = binomial
  )
}
```

```
if (exists("modelo_logistico")) {
  coeficientes_modelo <- summary(modelo_logistico)$coefficients
  head(coeficientes_modelo, 12)
}
```

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	18.107257168	101.84719297	0.17778848	8.588891e-01
MES02	-0.045765700	0.02975556	-1.53805548	1.240350e-01
MES03	-0.032191364	0.02927313	-1.09968980	2.714673e-01
MES04	-0.002838049	0.02945462	-0.09635327	9.232400e-01
MES05	-0.015408552	0.02906672	-0.53010979	5.960358e-01
MES06	-0.054753129	0.02983667	-1.83509526	6.649158e-02
MES07	-0.076296781	0.03035194	-2.51373620	1.194598e-02
MES08	-0.094049541	0.02997105	-3.13801278	1.700975e-03
MES09	-0.077109438	0.03015044	-2.55748998	1.054306e-02
MES10	-0.154152374	0.02979697	-5.17342485	2.298416e-07
MES11	-0.106385033	0.02950811	-3.60528073	3.118157e-04
MES12	-0.111888509	0.02959338	-3.78086333	1.562855e-04

Interpretación del resultado.

Se muestran solo los primeros coeficientes para no saturar el PDF. Los coeficientes indican cambios en el logit respecto a categorías de referencia.

4.11. Predicción de probabilidades

```
if (exists("modelo_logistico") && exists("prueba")) {
  prueba$probabilidad_victimas <- predict(
    modelo_logistico,
    newdata = prueba,
    type = "response"
  )

  head(prueba |> select(accidente_con_victimas, victimas_binaria, probabilidad_victimas))
}
```

A tibble: 6 x 3

accidente_con_victimas	victimas_binaria	probabilidad_victimas
<fct>	<dbl>	<dbl>

1 Solo daños	0	0.0685
2 Solo daños	0	0.116
3 Con víctimas	1	1.000
4 Solo daños	0	0.111
5 Solo daños	0	0.111
6 Solo daños	0	0.0632

Interpretación del resultado.

Cada valor de `probabilidad_victimas` estima la probabilidad de que el accidente tenga víctimas.

4.12. De probabilidades a clases

```
if (exists("prueba")) {
  prueba$prediccion_05 <- ifelse(
    prueba$probabilidad_victimas >= 0.5,
    "Con víctimas",
    "Solo daños"
  )

  prueba$prediccion_05 <- factor(prueba$prediccion_05, levels = c("Con víctimas", "Solo daños"))
}
```

4.13. Matriz de confusión

```
if (exists("prueba")) {
  matriz_confusion_05 <- table(
    Real = prueba$accidente_con_victimas,
    Predicho = prueba$prediccion_05
  )

  matriz_confusion_05
}
```

Real	Predicho	
	Con víctimas	Solo daños
Con víctimas	4886	15618
Solo daños	807	95877

Interpretación del resultado.

La diagonal contiene clasificaciones correctas. Los falsos negativos son accidentes con víctimas predichos como solo daños.

4.14. Métricas de evaluación

```
if (exists("matriz_confusion_05")) {
  VP <- matriz_confusion_05["Con víctimas", "Con víctimas"]
}
```

```

FN <- matriz_confusion_05["Con víctimas", "Solo daños"]
FP <- matriz_confusion_05["Solo daños", "Con víctimas"]
VN <- matriz_confusion_05["Solo daños", "Solo daños"]

metricas_05 <- data.frame(
  punto_corte = 0.5,
  exactitud = (VP + VN) / (VP + FN + FP + VN),
  sensibilidad = VP / (VP + FN),
  especificidad = VN / (VN + FP)
)

metricas_05
}

```

```

      punto_corte exactitud sensibilidad especificidad
1          0.5 0.8598406      0.238295    0.9916532

```

4.15. Cambiar el punto de corte

```

if (exists("prueba")) {
  prueba$prediccion_03 <- ifelse(
    prueba$probabilidad_victimas >= 0.3,
    "Con víctimas",
    "Solo daños"
  )
  prueba$prediccion_03 <- factor(prueba$prediccion_03, levels = c("Con víctimas", "Solo daños"))

  matriz_confusion_03 <- table(Real = prueba$accidente_con_victimas, Predicho = prueba$prediccion_03)
  matriz_confusion_03
}

```

	Predicho	
Real	Con víctimas	Solo daños
Con víctimas	13765	6739
Solo daños	14302	82382

```

if (exists("matriz_confusion_03")) {
  VP_03 <- matriz_confusion_03["Con víctimas", "Con víctimas"]
  FN_03 <- matriz_confusion_03["Con víctimas", "Solo daños"]
  FP_03 <- matriz_confusion_03["Solo daños", "Con víctimas"]
  VN_03 <- matriz_confusion_03["Solo daños", "Solo daños"]

  metricas_03 <- data.frame(
    punto_corte = 0.3,
    exactitud = (VP_03 + VN_03) / (VP_03 + FN_03 + FP_03 + VN_03),
    sensibilidad = VP_03 / (VP_03 + FN_03),
    especificidad = VN_03 / (VN_03 + FP_03)
  )
}

```

```
rbind(metricas_05, metricas_03)
}
```

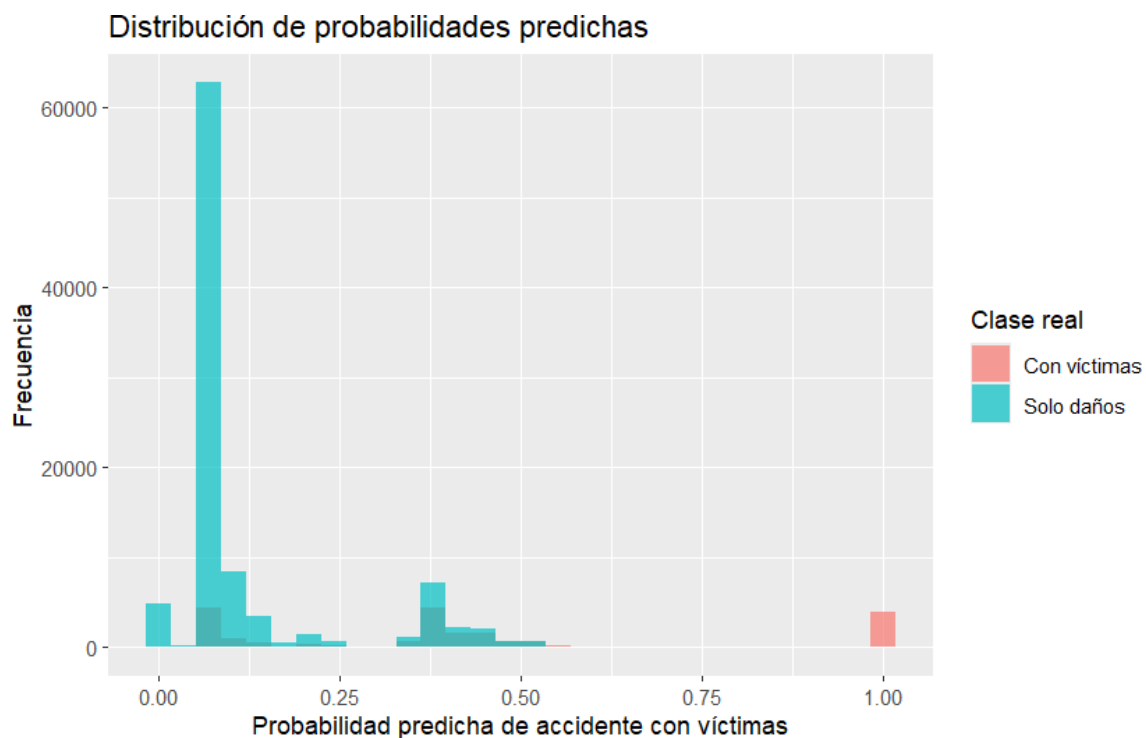
	punto_corte	exactitud	sensibilidad	especificidad
1	0.5	0.8598406	0.2382950	0.9916532
2	0.3	0.8204509	0.6713324	0.8520748

Interpretación del resultado.

Bajar el punto de corte suele aumentar la sensibilidad, aunque puede disminuir la especificidad. En seguridad vial, detectar más accidentes con víctimas puede ser más importante que maximizar la exactitud.

4.16. Distribución de probabilidades predichas

```
if (exists("prueba")) {
  ggplot(prueba, aes(x = probabilidad_victimas, fill = accidente_con_victimas)) +
    geom_histogram(bins = 30, alpha = 0.7, position = "identity") +
    labs(
      title = "Distribución de probabilidades predichas",
      x = "Probabilidad predicha de accidente con víctimas",
      y = "Frecuencia",
      fill = "Clase real"
    )
}
```



4.17. Odds ratios

```
if (exists("modelo_logistico")) {
  odds_ratios <- data.frame(
    variable = names(coef(modelo_logistico)),
    odds_ratio = exp(coef(modelo_logistico))
  )

  head(odds_ratios, 12)
}
```

	variable	odds_ratio
(Intercept)	(Intercept)	7.309402e+07
MES02	MES02	9.552658e-01
MES03	MES03	9.683213e-01
MES04	MES04	9.971660e-01
MES05	MES05	9.847096e-01
MES06	MES06	9.467188e-01
MES07	MES07	9.265412e-01
MES08	MES08	9.102377e-01
MES09	MES09	9.257885e-01
MES10	MES10	8.571414e-01
MES11	MES11	8.990784e-01
MES12	MES12	8.941439e-01

4.18. Conclusión

La regresión logística es un modelo útil, interpretable y adecuado para clasificación binaria. En este capítulo vimos que el punto de corte modifica el balance entre sensibilidad y especificidad.

Capítulo 5

k vecinos más cercanos, k-NN

5.1. Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de explicar k-NN, preparar datos para este método, estandarizar variables, construir una matriz de confusión y evaluar el desempeño.

5.2. Introducción

k-NN significa *k-Nearest Neighbors* o k vecinos más cercanos.

Para clasificar una nueva observación, buscamos las observaciones más parecidas a ella y le asignamos la clase más frecuente entre sus vecinos.

5.3. Idea intuitiva del método

El algoritmo k-NN calcula distancias, encuentra los k vecinos más cercanos, revisa sus clases y asigna la clase más frecuente.

5.4. Ejemplo sencillo con puntos

```
datos_ejemplo <- data.frame(  
  hora = c(8, 9, 10, 18, 19, 20, 22, 23),  
  mes = c(1, 1, 2, 6, 6, 7, 12, 12),  
  clase = c(  
    "Solo daños", "Solo daños", "Solo daños",  
    "Con víctimas", "Con víctimas", "Con víctimas",  
    "Con víctimas", "Con víctimas"  
  )  
)
```

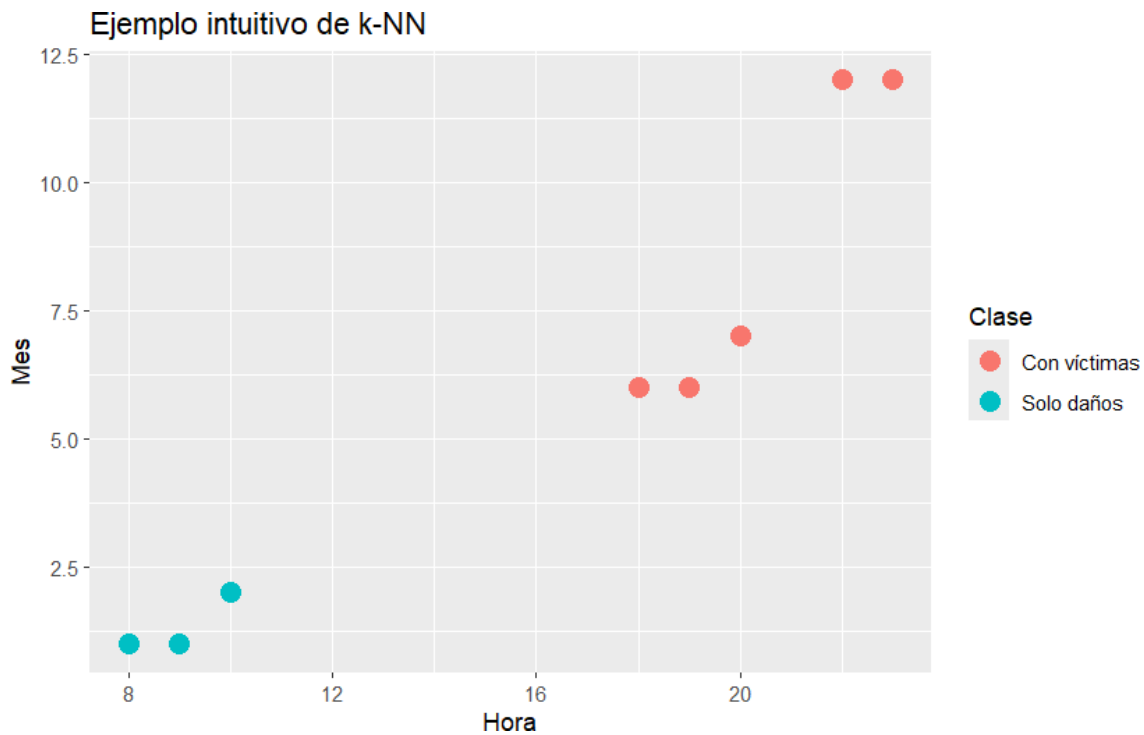
datos_ejemplo

hora	mes	clase
------	-----	-------

1	8	1	Solo daños
2	9	1	Solo daños
3	10	2	Solo daños
4	18	6	Con víctimas
5	19	6	Con víctimas
6	20	7	Con víctimas
7	22	12	Con víctimas
8	23	12	Con víctimas

```
library(ggplot2)

ggplot(datos_ejemplo, aes(x = hora, y = mes, color = clase)) +
  geom_point(size = 4) +
  labs(
    title = "Ejemplo intuitivo de k-NN",
    x = "Hora",
    y = "Mes",
    color = "Clase"
  )
```



Interpretación del resultado.

La gráfica muestra dos grupos separados. Una nueva observación se clasificaría según los puntos más cercanos.

5.5. Distancia euclidiana

Para dos puntos $A = (x_1, y_1)$ y $B = (x_2, y_2)$, la distancia euclidiana es:

$$d(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

En general, para p variables:

$$d(x_0, x_i) = \sqrt{\sum_{j=1}^p (x_{0j} - x_{ij})^2}$$

5.6. Ejemplo de cálculo de distancia

```
x1 <- 8
y1 <- 1

x2 <- 10
y2 <- 2

distancia <- sqrt((x1 - x2)^2 + (y1 - y2)^2)

distancia

[1] 2.236068
```

Interpretación del resultado.

Entre menor sea la distancia, más parecidas son las observaciones en las variables consideradas.

5.7. Estandarización de variables

Como k-NN depende de distancias, las variables deben estar en escalas comparables.

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

5.8. Algoritmo k-NN

Algoritmo: k vecinos más cercanos

Entrada:

- Matriz de entrenamiento X
- Clases de entrenamiento Y
- Nueva observación x_0
- Número de vecinos k

Proceso:

1. Estandarizar las variables predictoras.
2. Calcular distancias.
3. Ordenar distancias.

4. Seleccionar los k vecinos más cercanos.
5. Asignar la clase más frecuente.

Salida:

- Clase predicha

5.9. Cargar paquetes y datos

```
library(readr)
library(dplyr)
library(ggplot2)
library(class)
library(tidyr)

ruta_atus_ml <- "datos/atus_ml_preparado.csv"
file.exists(ruta_atus_ml)
```

[1] TRUE

```
if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
} else {
  atus_ml <- NULL
  print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo 2.")
}
```

[1] "Base preparada cargada correctamente."

5.10. Selección de una muestra

```
if (!is.null(atus_ml)) {
  set.seed(123)
  tamano_muestra <- min(12000, nrow(atus_ml))

  atus_knn_base <- atus_ml |>
    sample_n(tamano_muestra)

  data.frame(
    filas = nrow(atus_knn_base),
    columnas = ncol(atus_knn_base)
  )
}
```

	filas	columnas
1	12000	6

5.11. Preparar variables

```
if (exists("atus_knn_base")) {
  atus_knn_base <- atus_knn_base |>
  mutate(
    accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas", "Solo
    MES = as.factor(MES),
    ID_HORA = as.numeric(ID_HORA),
    DIASEMANA = as.factor(DIASEMANA),
    TIPACCID = as.factor(TIPACCID),
    CAUSAACCI = as.factor(CAUSAACCI)
  ) |>
  na.omit()

  round(100 * prop.table(table(atus_knn_base$accidente_con_victimas)), 2)
}
```

Con víctimas	Solo daños
17.36	82.64

Interpretación del resultado.

La muestra mantiene una clase mayoritaria: “Solo daños”. Por eso evaluaremos sensibilidad y especificidad, no solo exactitud.

5.12. Crear variables dummy

```
if (exists("atus_knn_base")) {
  matriz_predictoras <- model.matrix(
    ~ ID_HORA + MES + DIASEMANA + TIPACCID + CAUSAACCI - 1,
    data = atus_knn_base
  )

  y <- atus_knn_base$accidente_con_victimas

  data.frame(
    filas = nrow(matriz_predictoras),
    columnas = ncol(matriz_predictoras)
  )
}
```

	filas	columnas
1	12000	36

5.13. División en entrenamiento y prueba

```
if (exists("matriz_predictoras") && exists("y")) {
  set.seed(123)
  n <- nrow(matriz_predictoras)
  indices_entrenamiento <- sample(1:n, size = round(0.7 * n))

  x_entrenamiento <- matriz_predictoras[indices_entrenamiento, ]
  x_prueba <- matriz_predictoras[-indices_entrenamiento, ]

  y_entrenamiento <- y[indices_entrenamiento]
  y_prueba <- y[-indices_entrenamiento]

  data.frame(
    conjunto = c("Entrenamiento", "Prueba"),
    registros = c(nrow(x_entrenamiento), nrow(x_prueba))
  )
}
```

	conjunto	registros
1	Entrenamiento	8400
2	Prueba	3600

5.14. Estandarización

```
if (exists("x_entrenamiento") && exists("x_prueba")) {
  medias <- apply(x_entrenamiento, 2, mean)
  desviaciones <- apply(x_entrenamiento, 2, sd)
  desviaciones[desviaciones == 0] <- 1

  x_entrenamiento_esc <- scale(x_entrenamiento, center = medias, scale = desviaciones)
  x_prueba_esc <- scale(x_prueba, center = medias, scale = desviaciones)
}
```

5.15. Modelo k-NN con $k = 5$

```
if (exists("x_entrenamiento_esc") && exists("x_prueba_esc")) {
  pred_knn_5 <- knn(
    train = x_entrenamiento_esc,
    test = x_prueba_esc,
    cl = y_entrenamiento,
    k = 5
  )
}
```

5.16. Matriz de confusión

```
if (exists("pred_knn_5")) {
  pred_knn_5 <- factor(pred_knn_5, levels = c("Con víctimas", "Solo daños"))
  y_prueba <- factor(y_prueba, levels = c("Con víctimas", "Solo daños"))

  matriz_confusion_knn_5 <- table(
    Real = y_prueba,
    Predicho = pred_knn_5
  )

  matriz_confusion_knn_5
}
```

	Predicho	
Real	Con víctimas	Solo daños
Con víctimas	211	399
Solo daños	144	2846

5.17. Métricas de evaluación

```
calcular_metricas <- function(real, predicho) {
  real <- factor(real, levels = c("Con víctimas", "Solo daños"))
  predicho <- factor(predicho, levels = c("Con víctimas", "Solo daños"))

  matriz <- table(Real = real, Predicho = predicho)

  VP <- matriz["Con víctimas", "Con víctimas"]
  FN <- matriz["Con víctimas", "Solo daños"]
  FP <- matriz["Solo daños", "Con víctimas"]
  VN <- matriz["Solo daños", "Solo daños"]

  data.frame(
    exactitud = (VP + VN) / (VP + FN + FP + VN),
    sensibilidad = VP / (VP + FN),
    especificidad = VN / (VN + FP)
  )
}

if (exists("matriz_confusion_knn_5")) {
  metricas_knn_5 <- calcular_metricas(y_prueba, pred_knn_5)
  metricas_knn_5$k <- 5
  metricas_knn_5 |> select(k, exactitud, sensibilidad, especificidad)
}
```

	k	exactitud	sensibilidad	especificidad
1	5	0.8491667	0.3459016	0.9518395

5.18. Comparación con distintos valores de k

```
if (exists("x_entrenamiento_esc") && exists("x_prueba_esc")) {
  valores_k <- c(3, 5, 11)
  resultados_knn <- data.frame()

  for (k_actual in valores_k) {
    pred_actual <- knn(
      train = x_entrenamiento_esc,
      test = x_prueba_esc,
      cl = y_entrenamiento,
      k = k_actual
    )

    metricas_actuales <- calcular_metricas(y_prueba, pred_actual)
    metricas_actuales$k <- k_actual
    resultados_knn <- rbind(resultados_knn, metricas_actuales)
  }

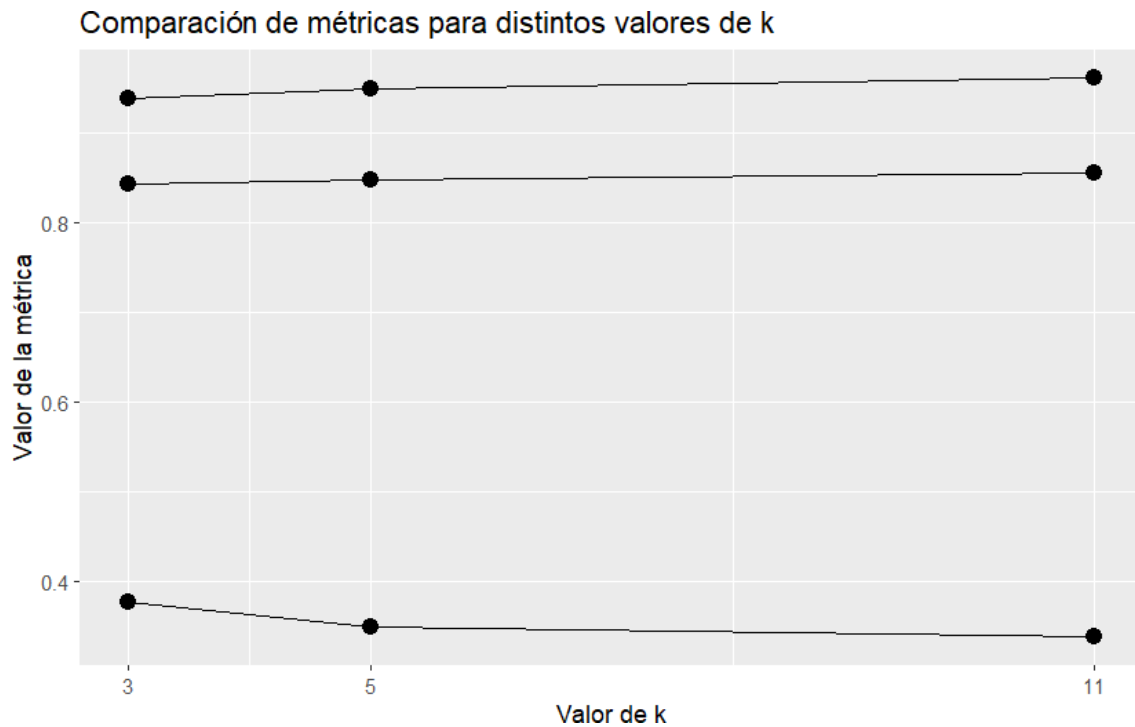
  resultados_knn <- resultados_knn |>
    select(k, exactitud, sensibilidad, especificidad)

  resultados_knn
}
```

	k	exactitud	sensibilidad	especificidad
1	3	0.8433333	0.3770492	0.9384615
2	5	0.8480556	0.3491803	0.9498328
3	11	0.8558333	0.3377049	0.9615385

```
if (exists("resultados_knn")) {
  resultados_largos <- resultados_knn |>
    pivot_longer(
      cols = c(exactitud, sensibilidad, especificidad),
      names_to = "metrica",
      values_to = "valor"
    )

  ggplot(resultados_largos, aes(x = k, y = valor, group = metrica)) +
    geom_line() +
    geom_point(size = 3) +
    scale_x_continuous(breaks = valores_k) +
    labs(
      title = "Comparación de métricas para distintos valores de k",
      x = "Valor de k",
      y = "Valor de la métrica"
    )
}
```

**Interpretación del resultado.**

Cambiar k modifica el equilibrio entre sensibilidad, especificidad y exactitud. No existe un valor universalmente mejor; debe elegirse según el objetivo del problema.

5.19. Comparación conceptual con regresión logística

Aspecto	Regresión logística	k-NN
Tipo de modelo	Paramétrico	No paramétrico
Produce modelo explícito	Sí	No en el mismo sentido
Usa distancias	No	Sí
Requiere escalamiento	No siempre	Sí
Interpretable	Alta	Media o baja

5.20. Conclusión

k-NN es un método intuitivo y útil para introducir clasificación basada en similitud. Su desempeño depende del valor de k , del escalamiento de las variables y de la calidad de las variables predictoras.

Capítulo 6

Árboles de decisión para clasificación

6.1. Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de:

- Explicar la idea básica de un árbol de decisión.
- Interpretar reglas de clasificación tipo “si-entonces”.
- Entrenar un árbol de decisión en R usando una base real.
- Generar predicciones sobre un conjunto de prueba.
- Construir una matriz de confusión.
- Calcular exactitud, sensibilidad y especificidad.
- Comparar conceptualmente árboles de decisión con regresión logística y k-NN.

6.2. Introducción

Los árboles de decisión son modelos de aprendizaje supervisado que clasifican observaciones mediante una secuencia de preguntas.

En este capítulo continuaremos con la base de accidentes de tránsito preparada en los capítulos anteriores. La variable que queremos predecir es:

`accidente_con_victimas`

Esta variable tiene dos clases:

- Con víctimas
- Solo daños

La idea central es construir reglas de decisión que permitan clasificar un accidente a partir de variables como mes, hora, día de la semana, tipo de accidente y causa presunta.

6.3. Idea intuitiva

Un árbol de decisión se parece a un diagrama de flujo. Cada nodo interno contiene una pregunta. Cada rama representa una posible respuesta. Cada hoja contiene una predicción.

Por ejemplo, un árbol podría formular preguntas como:

```
reglas_ejemplo <- data.frame(
  pregunta = c(
    "¿El accidente fue con peatón?",
    "¿Ocurrió de noche?",
    "¿La causa presunta fue el conductor?"
  ),
  posible_decision = c(
    "Mayor posibilidad de víctimas",
    "Revisar el patrón horario",
    "Analizar junto con el tipo de accidente"
  )
)

reglas_ejemplo
```

	pregunta	posible_decision
1	¿El accidente fue con peatón?	Mayor posibilidad de víctimas
2	¿Ocurrió de noche?	Revisar el patrón horario
3	¿La causa presunta fue el conductor?	Analizar junto con el tipo de accidente

Interpretación del resultado.

La tabla muestra ejemplos de preguntas que un árbol de decisión podría utilizar para separar accidentes con víctimas de accidentes de solo daños.

6.4. Reglas tipo si-entonces

Una ventaja de los árboles de decisión es que pueden expresarse como reglas sencillas.

Por ejemplo:

Si el tipo de accidente es colisión con peatón,
entonces aumenta la posibilidad de que el accidente tenga víctimas.

Si el accidente es una colisión con objeto fijo,
entonces el árbol revisa otras variables como la hora o la causa presunta.

Estas reglas hacen que los árboles sean útiles en contextos educativos, porque el estudiante puede seguir el razonamiento del modelo paso a paso.

6.5. Fundamento matemático

Un árbol de decisión busca dividir los datos en grupos más homogéneos. Para clasificación, una medida común de impureza es el índice de Gini.

$$Gini = 1 - \sum_{k=1}^K p_k^2$$

donde p_k representa la proporción de observaciones de la clase k dentro de un nodo.

En clasificación binaria, si un nodo contiene dos clases, el índice de Gini puede escribirse como:

$$Gini = 1 - p_1^2 - p_2^2$$

Un valor pequeño indica que el nodo es más puro. Un valor mayor indica que las clases están más mezcladas.

6.6. Ejemplo sencillo del índice de Gini

Supongamos que un nodo contiene 80 accidentes de solo daños y 20 accidentes con víctimas.

```
p_solo_danios <- 80 / 100
p_con_victimas <- 20 / 100

gini <- 1 - p_solo_danios^2 - p_con_victimas^2
gini
```

```
[1] 0.32
```

Interpretación del resultado.

El índice de Gini mide qué tan mezcladas están las clases dentro de un nodo. Si todas las observaciones fueran de una sola clase, el valor sería cero.

6.7. Algoritmo general

Algoritmo: Árbol de decisión para clasificación

Entrada:

- Base de datos con variables predictoras.
- Variable respuesta categórica.

Proceso:

1. Elegir una variable para dividir los datos.
2. Seleccionar la división que produce grupos más homogéneos.
3. Repetir el proceso en cada nuevo grupo.
4. Detener el crecimiento del árbol mediante criterios de control.
5. Asignar una clase a cada hoja.
6. Evaluar el modelo con datos de prueba.

Salida:

- Árbol entrenado.
- Reglas de decisión.
- Predicciones.
- Métricas de evaluación.

6.8. Cargar paquetes y datos

```
library(readr)
library(dplyr)
library(rpart)
```

```
ruta_atus_ml <- "datos/atus_ml_preparado.csv"
file.exists(ruta_atus_ml)
```

```
[1] TRUE
```

```
if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
} else {
  atus_ml <- NULL
  print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo 2.")
}
```

```
[1] "Base preparada cargada correctamente."
```

Interpretación del resultado.

Si aparece el mensaje de carga correcta, podemos continuar con el entrenamiento del árbol de decisión.

6.9. Usar la base completa

En este capítulo usaremos la base completa. Para evitar que el árbol crezca demasiado, controlaremos su complejidad durante el entrenamiento.

```
if (!is.null(atus_ml)) {
  atus_arbol_base <- atus_ml
  data.frame(
    filas = nrow(atus_arbol_base),
    columnas = ncol(atus_arbol_base)
  )
}
```

```
      filas columnas
1 390627         6
```

Interpretación del resultado.

Se conserva toda la base preparada. El control del tamaño del árbol se realizará mediante parámetros del modelo, no reduciendo el número de registros.

6.10. Preparar variables

```
if (exists("atus_arbol_base")) {
  atus_arbol_base <- atus_arbol_base |>
  mutate(
```

```

    accidente_con_victimas = factor(
      accidente_con_victimas,
      levels = c("Con víctimas", "Solo daños")
    ),
    MES = as.factor(MES),
    ID_HORA = as.numeric(ID_HORA),
    DIASEMANA = as.factor(DIASEMANA),
    TIPACCID = as.factor(TIPACCID),
    CAUSAACCI = as.factor(CAUSAACCI)
  ) |>
  na.omit()

  table(atus_arbol_base$accidente_con_victimas)
}

```

Con víctimas	Solo daños
68108	322519

```

if (exists("atus_arbol_base")) {
  round(100 * prop.table(table(atus_arbol_base$accidente_con_victimas)), 2)
}

```

Con víctimas	Solo daños
17.44	82.56

Interpretación del resultado.

La base está desbalanceada: la clase Solo daños es mayoritaria. Por eso no conviene evaluar el modelo solo con exactitud.

6.11. División en entrenamiento y prueba

```

if (exists("atus_arbol_base")) {
  set.seed(123)

  n <- nrow(atus_arbol_base)
  indices_entrenamiento <- sample(1:n, size = round(0.7 * n))

  entrenamiento <- atus_arbol_base[indices_entrenamiento, ]
  prueba <- atus_arbol_base[-indices_entrenamiento, ]

  data.frame(
    conjunto = c("Entrenamiento", "Prueba"),
    registros = c(nrow(entrenamiento), nrow(prueba)),
    porcentaje = c(
      round(100 * nrow(entrenamiento) / nrow(atus_arbol_base), 2),
      round(100 * nrow(prueba) / nrow(atus_arbol_base), 2)
    )
  )
}

```

```

    )
  )
}

```

	conjunto	registros	porcentaje
1	Entrenamiento	273439	70
2	Prueba	117188	30

Interpretación del resultado.

El conjunto de entrenamiento se usa para construir el árbol. El conjunto de prueba se usa para evaluar el desempeño con datos no utilizados durante el entrenamiento.

6.12. Ajustar el árbol de decisión

El árbol se ajusta con toda la base de entrenamiento, pero se limita su crecimiento para evitar sobreajuste y para facilitar la generación del libro en HTML y PDF.

```

if (exists("entrenamiento")) {
  modelo_arbol <- rpart(
    accidente_con_victimas ~ MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
    data = entrenamiento,
    method = "class",
    control = rpart.control(
      cp = 0.005,
      minsplit = 1000,
      minbucket = 300,
      maxdepth = 5
    )
  )
}

```

Interpretación del resultado.

El modelo queda entrenado. Los parámetros de control impiden que el árbol crezca de forma excesiva.

6.13. Tabla de complejidad

```

if (exists("modelo_arbol")) {
  printcp(modelo_arbol)
}

```

Classification tree:

```

rpart(formula = accidente_con_victimas ~ MES + ID_HORA + DIASEMANA +
      TIPACCID + CAUSAACCI, data = entrenamiento, method = "class",
      control = rpart.control(cp = 0.005, minsplit = 1000, minbucket = 300,
                              maxdepth = 5))

```

Variables actually used in tree construction:

```
[1] TIPACCID
```

```
Root node error: 47604/273439 = 0.17409
```

```
n= 273439
```

	CP	nsplit	rel error	xerror	xstd
1	0.097051	0	1.0000	1.0000	0.0041653
2	0.005000	2	0.8059	0.8059	0.0038150

Interpretación del resultado.

La tabla de complejidad muestra cómo cambia el error del árbol conforme se permiten más divisiones.

6.14. Importancia de variables

```
if (exists("modelo_arbol")) {
  importancia <- modelo_arbol$variable.importance

  if (!is.null(importancia)) {
    importancia_df <- data.frame(
      variable = names(importancia),
      importancia = as.numeric(importancia),
      row.names = NULL
    )

    importancia_df <- importancia_df[order(-importancia_df$importancia), ]
    importancia_df
  }
}
```

	variable	importancia
1	TIPACCID	22919.330
2	CAUSAACCI	1093.984

Interpretación del resultado.

Las variables con mayor importancia son las que más contribuyen a separar las clases dentro del árbol.

6.15. Gráfica opcional del árbol

La gráfica completa del árbol puede ser pesada al generar PDF. Por eso se deja como código opcional. El lector puede ejecutarla manualmente en RStudio.

```
plot(modelo_arbol, uniform = TRUE, margin = 0.1)
text(modelo_arbol, use.n = TRUE, cex = 0.7)
```

6.16. Predicción sobre el conjunto de prueba

```
if (exists("modelo_arbol") && exists("prueba")) {
  pred_arbol <- predict(
    modelo_arbol,
    newdata = prueba,
    type = "class"
  )

  head(pred_arbol)
}
```

1	2	3	4	5	6
Solo daños	Solo daños	Con víctimas	Solo daños	Solo daños	Solo daños

Levels: Con víctimas Solo daños

Interpretación del resultado.

Cada valor predicho corresponde a la clase que el árbol asigna a una observación del conjunto de prueba.

6.17. Matriz de confusión

```
if (exists("pred_arbol")) {
  pred_arbol <- factor(pred_arbol, levels = c("Con víctimas", "Solo daños"))
  real_arbol <- factor(prueba$accidente_con_victimas, levels = c("Con víctimas", "Solo daños"))

  matriz_confusion_arbol <- table(
    Real = real_arbol,
    Predicho = pred_arbol
  )

  matriz_confusion_arbol
}
```

	Predicho	
Real	Con víctimas	Solo daños
Con víctimas	3965	16539
Solo daños	0	96684

Interpretación del resultado.

La diagonal principal contiene los aciertos. Los valores fuera de la diagonal representan errores de clasificación.

6.18. Métricas de evaluación

```
calcular_metricas_arbol <- function(real, predicho) {
  real <- factor(real, levels = c("Con víctimas", "Solo daños"))
```

```

predicho <- factor(predicho, levels = c("Con víctimas", "Solo daños"))

matriz <- table(Real = real, Predicho = predicho)

VP <- matriz["Con víctimas", "Con víctimas"]
FN <- matriz["Con víctimas", "Solo daños"]
FP <- matriz["Solo daños", "Con víctimas"]
VN <- matriz["Solo daños", "Solo daños"]

data.frame(
  exactitud = (VP + VN) / (VP + FN + FP + VN),
  sensibilidad = VP / (VP + FN),
  especificidad = VN / (VN + FP)
)
}

if (exists("pred_arbol")) {
  metricas_arbol <- calcular_metricas_arbol(real_arbol, pred_arbol)
  metricas_arbol
}

```

```

      exactitud sensibilidad especificidad
1 0.8588678      0.1933769              1

```

Interpretación del resultado.

La exactitud mide la proporción total de clasificaciones correctas. La sensibilidad mide la capacidad para detectar accidentes con víctimas. La especificidad mide la capacidad para detectar accidentes de solo daños.

6.19. Probabilidades predichas

```

if (exists("modelo_arbol") && exists("prueba")) {
  probabilidades_arbol <- predict(
    modelo_arbol,
    newdata = prueba,
    type = "prob"
  )

  head(probabilidades_arbol)
}

```

```

      Con víctimas Solo daños
1  0.07433786  0.9256621
2  0.07433786  0.9256621
3  1.00000000  0.0000000
4  0.07433786  0.9256621
5  0.07433786  0.9256621
6  0.07433786  0.9256621

```

Interpretación del resultado.

El árbol también puede producir probabilidades estimadas de pertenencia a cada clase.

6.20. Resumen de nodos del árbol

Para evitar salidas demasiado largas, mostramos solo los primeros nodos del árbol.

```
if (exists("modelo_arbol")) {
  frame_arbol <- modelo_arbol$frame
  resumen_nodos <- data.frame(
    nodo = row.names(frame_arbol),
    variable = frame_arbol$var,
    n = frame_arbol$n,
    perdida = frame_arbol$dev,
    yval = frame_arbol$yval,
    row.names = NULL
  )

  head(resumen_nodos, 10)
}
```

	nodo	variable	n	perdida	yval
1	1	TIPACCID	273439	47604	2
2	2	TIPACCID	65214	32125	2
3	4	<leaf>	9240	0	1
4	5	<leaf>	55974	22885	2
5	3	<leaf>	208225	15479	2

Interpretación del resultado.

La tabla muestra algunos nodos del árbol. Los nodos marcados como <leaf> son hojas, es decir, puntos finales donde se asigna una clase.

6.21. Comparación conceptual con otros métodos

Aspecto	Regresión logística	k-NN	Árbol de decisión
Tipo de modelo	Paramétrico	No paramétrico	No paramétrico
Interpretabilidad	Alta	Media o baja	Alta
Usa distancias	No	Sí	No
Requiere escalamiento	No siempre	Sí	No
Produce reglas explícitas	Parcialmente	No	Sí
Maneja no linealidad	Limitado	Sí	Sí

6.22. Ventajas y desventajas**6.22.1. Ventajas**

- Son fáciles de explicar.

- Generan reglas claras.
- No requieren estandarizar variables.
- Pueden manejar relaciones no lineales.
- Funcionan con variables numéricas y categóricas.

6.22.2. Desventajas

- Pueden sobreajustarse si crecen demasiado.
- Pequeños cambios en los datos pueden modificar la estructura del árbol.
- Un solo árbol puede tener menor desempeño que métodos de ensamble como Random Forest.

6.23. Conclusión

Los árboles de decisión son modelos intuitivos, visuales e interpretables. Permiten clasificar observaciones mediante reglas tipo si-entonces.

En este capítulo usamos un árbol de decisión para clasificar accidentes de tránsito según la presencia de víctimas. También calculamos una matriz de confusión y métricas de evaluación.

Este capítulo prepara el camino para estudiar modelos más potentes basados en muchos árboles, como Random Forest.

